

Received 20 December 2024, accepted 4 January 2025, date of publication 10 January 2025, date of current version 17 January 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3528257

RESEARCH ARTICLE

Time-Sensitive Smart Cities: Addressing Smart City Challenges With TSN

RUI EDUARDO LOPES¹, (Member, IEEE), DUARTE RAPOSO², (Member, IEEE),
RODRIGO MARTINS², (Member, IEEE), AND SUSANA SARGENTO¹, (Member, IEEE)

¹Instituto de Telecomunicações, Universidade de Aveiro, Campus Universitário de Santiago, 3810-193 Aveiro, Portugal

²Instituto de Telecomunicações, Campus Universitário de Santiago, 3810-193 Aveiro, Portugal

Corresponding author: Rui Eduardo Lopes (ruieduardo.fa.lopes@ua.pt)

This work was supported in part by the Fundação para a Ciência e Tecnologia under Grant FCT.2021.06223.BD (DOI: 10.54499/2021.06223.BD), and in part by the PRR–Plano de Recuperação e Resiliência and the NextGenerationEU Funds at University of Aveiro, through the scope of the Agenda for Business Innovation “NEXUS: Pacto de Inovação–Transição Verde e Digital para Transportes, Logística e Mobilidade” under Grant C645112083-00000059.

ABSTRACT As cities transition into interconnected hubs of technology, the need for efficient time management across diverse interconnected devices becomes a requirement for supporting critical services for emergency, traffic, and other use cases. Time-sensitive network (TSN) emerges as a promising solution, offering standards and mechanisms to synchronize and regulate time-critical traffic in smart city infrastructures. This study researches the main use cases, key standards and mechanisms of TSN, evaluating their feasibility and adaptability to the heterogeneous equipment present in an urban environment of a smart city infrastructure. Through experimental assessments, this article demonstrates the efficacy of TSN-based solutions and also discusses hardware enhancements that complement TSN protocols, improving precision and resource utilization to ensure criticality in urban connectivity, enabling precise control over traffic conditions and service delivery towards the future smart city, as an environment of interconnected things. Moreover, we propose a classification of nine traffic categories to encompass the various traffic flows in a smart city deployment, and validate the application of time-aware and credit-based shaping of all nodes in the network according to the type of service. Results show that TSN is a valid choice to manage mixed-criticality of services in a smart city.

INDEX TERMS Time-sensitive networks (TSN), deterministic, reliability, resource reservation, mixed-criticality, survey, smart cities, Industry 4.0, automotive.

I. INTRODUCTION

Societies throughout the world evolve on a daily basis and, with it, their living environments such as cities accompany the same trend. Within this behavior, cities are augmenting themselves from “inhabited places of greater size, population, or importance than a town or a village”¹ to environments plentiful of technology such as autonomous things, and of “interconnected nodes where traditional networks and

services are made more efficient with the use of digital solutions for the benefit of its inhabitants and businesses”².

These are Smart Cities, and with this constant evolution of use cases for autonomy and interconnection of heterogeneous nodes that comprises such a network, the amount of network traffic necessities also enlarges, over which one should reason on the relevance of time as a variable and how can it be controlled over this large set of devices.

Although being the most natural variable we deal everyday with, time is never considered as a priority in the context

The associate editor coordinating the review of this manuscript and approving it for publication was Derek Abbott¹.

¹As quoted from Merriam-Webster.com Dictionary, Merriam-Webster, <https://www.merriam-webster.com/dictionary/city>. Accessed 10 Sep. 2024

²From European Union’s concept of Smart Cities, as mentioned in https://commission.europa.eu/eu-regional-and-urban-development/topics/cities-and-urban-development/city-initiatives/smart-cities_en. Accessed 10 Sep. 2024

of computer networks. Rather than, we design our systems to achieve the best results possible in time, such as placing our efforts into acquiring the *lowest latency possible*, for instance. But in more specialized contexts, such as industrial contexts where other requirements such as stability and reliability exist, there are already some solutions that were created throughout the years. Within those alternatives for handling time within a network and considering those non-proprietary, time-sensitive networks (TSNs) are brought as a toolbox of possible solutions for their ability of handling mixed-criticality scenarios, in which in this document we duly analyze by its standards and mechanisms, as well as how can it be used within this pioneering smart city context or profile. TSN approaches offer standards and mechanisms to synchronize and regulate time-critical traffic, enabling critical services for emergency, traffic, and other use cases within the context of a smart city, by the ability of efficiently manage time between diverse interconnected infrastructure devices [1], [2].

In this article we propose the usage of TSN to build up smart city solutions with time constraints and quality of service characteristics in mind, corroborating with experiments on some of its standards to assess how feasible these approaches are in this new use case for TSN. In summary we contribute to the following questions:

- 1) What are the main standards and mechanisms of TSN and how can they be useful in a Smart City?
- 2) How to study requirements and apply TSN time synchronization and latency standards in a Smart City?
- 3) What are the main use cases for TSN and how can a Smart City take part of it?
- 4) How versatile and robust are TSN standards in regards to the heterogeneous equipment in a Smart City?

Within this article, we explain the key pillars of TSN, directly intended for smart city researchers wanting to improve their understanding of this toolbox of standards and mechanisms. This approach of TSN in smart cities and contribution, to the best of our knowledge, offers a novel viewing of a TSN setting as well as a set of new tools to better organize network service traffic within a smart city infrastructure, placing time as a top-level consideration within the set of city flows.

This article proceeds, in section II, with a brief definition of TSN standards and concepts within the context of a smart city and its requirements. Following this description, in section III a contextualization of how and where is TSN commonly used is described, allowing to bridge out to a smart city superset in section IV where this environment is regarded as a TSN deployable space. Section V mentions and reports some experiments performed with considerations for both time-critical and bandwidth-bound traffic, with an ending discussion of results including a depiction of some hardware benefits to the base TSN standards used. Ending this document, both a conclusion and future work are presented.

II. TIME-SENSITIVE NETWORKS CORE STANDARDS

Natively and throughout history, common network equipment does not have any concept of “time” to provide synchronization features and precision timing in packet/frame processing and handling. As history led packets to be more prioritized regarding delivery ratios, rather than within a specific time range, network nodes are optimally made to reduce such criteria. With these pieces of equipment, despite each node has reduced (with optimization) the processing delay in packet hops, the total delay introduced by intermediate nodes in an average packet flow path can be unacceptably high.

A. A TIME-SENSITIVE NETWORK (TSN) INTRODUCTION

Some of the evolution made in history to solve these issues was led by transport-level protocols, which based their behaviors on throttling and retransmitting packets to prevent networking congestion at such a level. Lacking congestion prevention at the link layer (layer 2 of the Open Systems Interconnection (OSI) communication model), the Institute of Electrical and Electronics Engineers (IEEE) 802.1 working group³ is researching, developing, and standardizing a set of protocols to tackle such issues, in a technology task group named time-sensitive network (TSN).

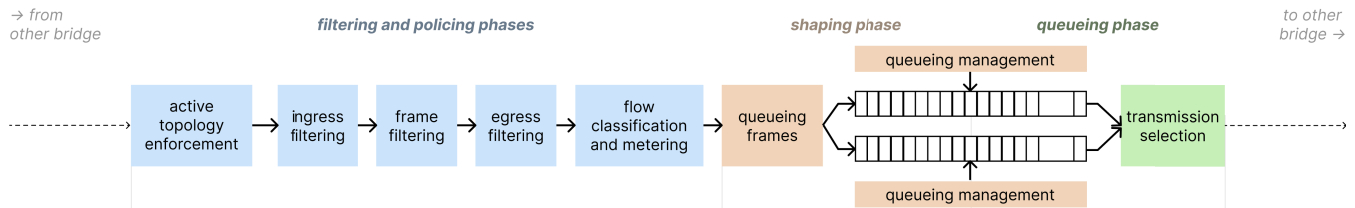
With this idea taken from industrial applications, Ethernet plays an increasingly critical role in communications with high data rates, low implementation and deployment costs, and easy integration with other equipment. Nevertheless, and especially in the context of industrial applications (which are, in a big part of it, considered critical software applications), we enter a time-critical environment. Although Ethernet allows one to develop such time-critical applications allowing multiple traffic classes to be determined and used according to the characteristics that quality-of-service (QoS) protocols specify, TSN attempts to provide additional standards to limit end-to-end delays and jitter by introducing queueing techniques and scheduling mechanisms into action [4].

TSN is considered to be a set of tools towards a *deterministic approach*, that is, a solution of a set of deterministic data paths for real-time applications with meager data loss rates, packet delay variation, and bounded latency. In other words, in a traditional service, lowering the latency meant, in the average of cases, losing packets or overprovisioning; however, with deterministic considerations, one can state that the correct packet is to be always sent at the right time.

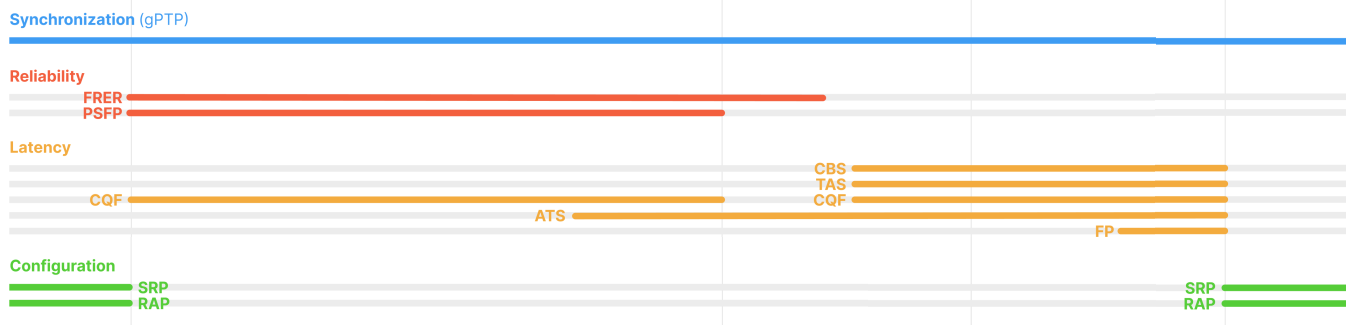
However, TSN is not intended to meet all real-time requirements due to variations in service criteria, meaning is not a one size fits all solution. In specific contexts like automotive scenarios, certain subsystems like ADAS sensors require precise timing down to microseconds, unlike more flexible criteria for subsystems like infotainment. This distinction was highlighted by the IEEE’s TSN working group in 2018, as they sought to integrate the existing audio/video bridging (AVB) standards with the then newly designated TSN. TSN was augmented with improved time synchro-

³<https://1.ieee802.org>

(a) Packet forwarding process steps in a bridge (as in IEEE 802.1Q)



(b) TSN Categories and some of its protocols and mechanisms' impact throughout the forwarding process

**FIGURE 1.** Forwarding process of a TSN bridge relative to scope ranges of TSN standards, as defined in IEEE 802.1Q [3].

nization models (featuring fault-tolerant, highly accurate clock synchronization) and real-time models (allowing time shaping instead of just adjusting bounded delay of data streams). Additionally, TSN incorporated reliability features (enabling system redundancy) and configuration options. While this document does not delve into the AVB versus TSN differentiation, Deng et al. in [5] provide a comprehensive reference on the design and modeling of real-time Ethernet.

In summary, TSN, rather than a protocol or a mechanism to implement time-critical specifications, is a toolbox which provides a wide variety of functions to work on. From filtering purposes to policing, shaping or queueing tasks, TSN covers the possibility of selecting one or more of these functions in order to gather more control of time-critical requirements for a given or multiple data flows. As originally specified by the TSN working group of IEEE, TSN is inherently comprised of four pillars: a) time synchronization; b) high availability and ultra reliability; c) bounded low latency; and d) dedicated resource management. In each one of these topics, several subtopics are explicit, which coincide with the protocols that make up TSN.

Under the TSN concept, the topology is comprised of bridges and end-stations. An end-station can be classified as a talker, if it is a source or a producer of a data stream; or as a listener, if it is a destination, receiver, or a consumer of a stream. On the other hand, a bridge is a suitable switching device, which has an architecture defined in IEEE 802.1Q [3].

From this knowledge, the TSN workgroup has worked in several mechanisms and protocols with which the steps from the reception of a frame to a transmission, are performed in parallel to best achieve time criteria of TSN communications, which the following subsection will detail.

In order to adjust our reasoning within the scope of a solution using TSN, we first will briefly explain the key mechanisms and standards that build up the TSN toolset. The following subsections aim to be a concise and useful resource mostly for contextualization for whom never was introduced to these standards. We will commence by presenting an overview of the primary TSN standards depicted in figure 1, delving into their conceptual frameworks and research studies.

B. TIME SYNCHRONIZATION MAIN FEATURES

Commencing with time synchronization, and as defined in IEEE 802.1AS-Rev, the primary clock synchronization protocol utilized globally across all nodes within a TSN network is the generalized precision time protocol (gPTP). This protocol gauges the time needed for receiving, processing, queuing, and transmitting data at each bridge, spanning from ingress to egress ports, along with the link latency for each hop. Subsequently, this time metric is compared with the time reference dictated by a central clock entity known as the grand master clock (GM). The GM is determined beforehand through the execution of a best master clock (BMC) algorithm, resembling the spanning tree protocol, aiming to synchronize all nodes in the network (see figure 2), hierarchically from the master to its slaves, here depicted within a city context [6].

The bridges with gPTP send timing correction data based through the connection and the bridge itself. The devices periodically send a `sync` message and a correcting `follow-up` message from the master port (clock master port) to the slave port(s) (clock slave ports). This allows nanosecond precision, unlike solutions such as the network

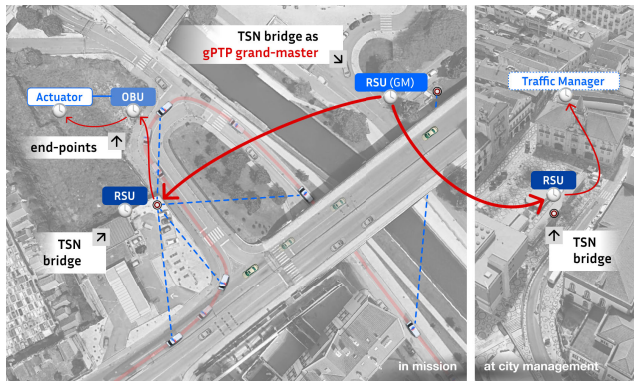


FIGURE 2. Generalized precision time protocol (gPTP) clocking hierarchy, from road-side units (RSUs) to on-board units (OBUs).

time protocol (NTP), and it is essential for other TSN standards.

There is more to time synchronization in TSN besides gPTP, regarding different network access technologies such as 5G [7], [8], but also regarding new amendments to IEEE 802.1AS so to specify a hot standby protocol, in which it includes functions for converting and directing synchronization time between gPTP domains, and determining domain quality [9], [10].

C. RELIABILITY MAIN FEATURES

Stepping into the **ultra reliability and high availability** features of TSN, we are now embracing issues of transmission errors, delivery guarantees, redundancies and fault tolerances.

1) FRAME REPLICATION AND ELIMINATION FOR RELIABILITY (FRER)

Starting with the frame replication and elimination for reliability (FRER) standard, defined in the IEEE 802.1CB [11], this function allows TSN bridges and end-nodes to send duplicate copies of each frame over multiple disjoint paths along the way towards a single or multiple listeners. While granting redundancy in the network, its usage takes profit of the transmitted flows traffic class and path information to minimize the network congestion, as each copied frame has a proper sequence identification number, used to reorder and merge frames, and to discard duplicates.

With FRER, both the end-points of a given flow and the TSN bridges are able either to replicate or to eliminate duplicates, as shown in figure 3, where three cases are shown with such a variety of options being depicted.

Note that FRER does not recover any lost packet, neither uses any encoding procedure to recover lost frames, but creates redundancy in the system to diminish the probability of lost packets. This means that even if any of the connections where replicated frames are encountered have a link failure, no data will be missed, as long as there is at least one available link.

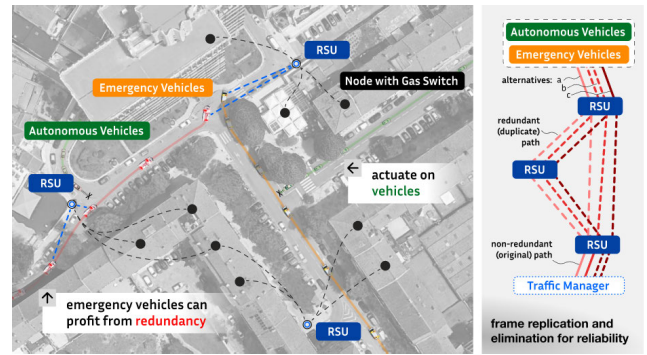


FIGURE 3. Frame replication and elimination for redundancy (FRER).

2) PER-STREAM FILTERING AND POLICING (PSFP)

Another feature for reliability in TSN is the per-stream filtering and policing (PSFP), as defined in the IEEE 802.1Qci standard [12] (figure 5). As the name already states, this feature works within the filtering and policing phases of the forwarding process in a bridge, as depicted in figure 1. PSFP support requires that rules to be applied are within the scope of three instance tables whose evaluation happens before any queuing procedures happen: (i) a *stream filter table*, where filtering and policing actions are applied to frames received on a specific stream; (ii) a *stream gate table*, where frames of a given stream are allowed to pass (or not) through the stream's gate; and (iii) a *flow meter table*, which is responsible to perform statistics and metering on each flow entering a bridge.

As its main function is to protect streams against bandwidth violations, malfunctioning, and malicious attacks, it works right at the reception of the frames, in a per-stream-oriented fashion.

3) SUMMARY

The current reliability-related features in TSN can have a notable and beneficial impact on traffic management and redundancy, ultimately enhancing a system's availability and reliability. The diagram presented in figure 4 provides a visual representation of how these reliability features affect the forwarding process of a bridge, following the guidelines of the IEEE 802.1Q standard.

The reliability features of TSN encompass a range of tools that have the potential to enhance the security functionalities and communication resilience of a city's infrastructure. These tools enable the identification, replication, and elimination of frames by FRER, implement both ingress and egress policies on each bridge, or facilitate stream filtering and metering of flow packets, among other features. Despite being a brief description of what TSN currently has to offer in regard to reliability, a more detailed tutorial on their behaviors could be found in the following articles and surveys:

- Seol et al. provided a thorough description of the impact of FRER as a central reliability feature in

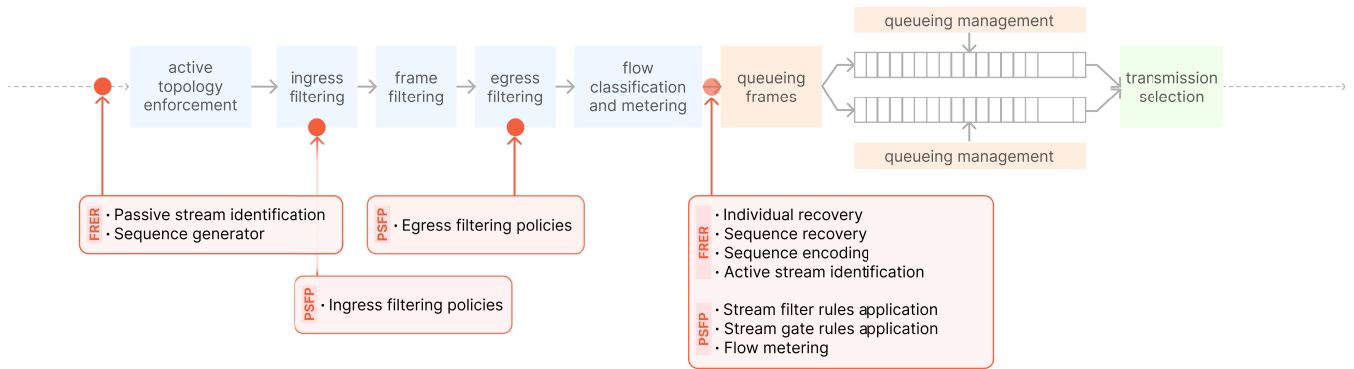


FIGURE 4. TSN's main reliability features' impact in detail, along the course of the forwarding process of a bridge.



FIGURE 5. Per-stream filtering and policing (PSFP).

TSN in their work [13]. They also referenced the study by Kehrer et al. [14], offering a comprehensive comparison of FRER with other redundancy protocols. We also recommend referring to this work for a deeper understanding of this concept.

- Deng et al., in [5], exposes a more detailed criteria on how to model both reliability and security for TSN using FRER, path control and reservation (PCR), and PSFP.
- Grigorjew et al., in [15] takes an overall systems viewpoint of a TSN-based solution and deeply analyses the resilience provided by the interaction of multiple running mechanisms. This work does not exclusively focus on synchronization issues, but highlights the meaning of resilience as a transversal concept to TSN approaches.

D. LATENCY MAIN FEATURES

Shifting our focus to **latency-related standards** within the TSN suite, the majority of these standards are primarily relative to shaping features of the flows intended for forwarding through a bridge. Essentially, with the exception of one standard, all latency-aware TSN features operate under the assumption that all processing related to filtering and policing has already been completed at this execution stage.

1) CREDIT-BASED SHAPER (CBS)

Starting with the credit-based shaper (CBS), as defined in IEEE 802.1Qav [16], it serves as a bandwidth-oriented shaper allocated immediately after specific queues. It restricts the throughput of a queue using a defined slope, measured in data units per second. This throughput is dynamic and corresponds to a given amount of credits associated with this shaper on a per-queue basis. Such a credit is increased if frames are waiting for other queues to be transmitted (at a given idle slope rate), and is decreased during transmission, at a rate called send slope. This credit is the reason for opening and closing each queue gates before transmission selection, whose selects frames by the same order as they entered queues in the queueing phase [3]. This functionality is instrumental in shaping and smoothing traffic to prevent starvation and minimize jitter.

2) TIME-AWARE SHAPER (TAS)

Next, the time-aware shaper (TAS), as defined in IEEE 802.1Qbv [17], represents a shaper that prioritizes transmission queues' time over bandwidth. With consistent clock synchronization across the network, by the help of gPTP features as mentioned earlier, the TAS operates a shaper whose primary components are gates that are positioned right after each one of the queues. The state of these gates is controlled by a gate control list (GCL). The GCL contains a schedule conducting the opening of the queues' gates within the time cycles defined in the TAS procedure configuration. This shaper is designed to characterize the connection in fixed length and repeating time cycles, considering queues having a proper Ethernet priority and being able to be granted exclusive use of the egress port to transmit their content, as this coordination is strictly configured by a set of rules in the gate control list, which is synchronized with the bridge's clock.

In figure 6, a simple representation combining both CBS and TAS for a smart city scenario can be seen. This depiction attempts to bring some context to the combination by creating a pipeline with both shapers and allowing

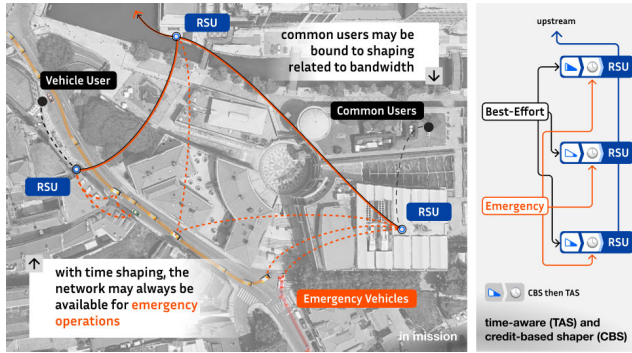


FIGURE 6. Combination of CBS and TAS shapers in a single latency handling pipeline within the context of a Smart City application.

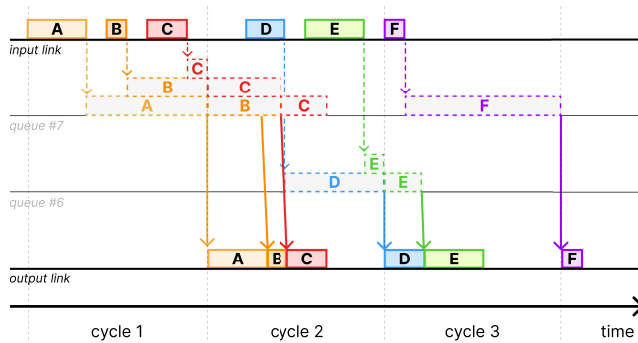


FIGURE 7. Cyclic queueing and forwarding (CQF).

them to be served in different instants conforming to the type of traffic. Being this example within the context of a smart city, we must be able to prioritize emergency traffic against other best-effort flows, commonly characterized by being the network profile defining common users usage. Starting with a first layer of shaping with CBS, this latter type of traffic could be bound to shaping, so to allow emergency flows to have a larger room for transmission. As this is not enough to guarantee the transmission of the emergency events, with TAS being performed as a second part of our shaping pipeline, all queues are subject to a transmission schedule over which time slots can be designed to open queues' gates specifically for emergency traffic flows.

3) CYCLIC QUEUEING AND FORWARDING (CQF)

By comprehending the functioning of TAS and leveraging the reliability feature of PSFP, one can gain insights into the inner workings of the cyclic queueing and forwarding (CQF) feature in TSN (depicted in figure 7). This feature, defined in IEEE 802.1Qch [18], segments the transmission time into time intervals of equal duration, where frames received in one interval are forwarded in the next. Similar to TAS, this feature requires robust global clock synchronization to provide minimal jitter. It is often referred to as a peristaltic shaper due to its cyclic nature.

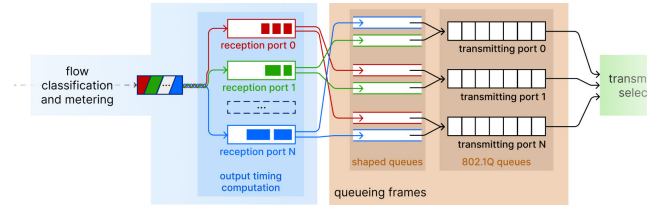


FIGURE 8. Asynchronous traffic shaper (ATS). Adapted from Specht, 2016⁴.

4) ASYNCHRONOUS TRAFFIC SHAPER (ATS)

Expanding this category's scope of action, the asynchronous traffic shaper (ATS) standard, described in IEEE 802.1Qcr [19], sets itself apart from the previously mentioned standards within TSN (except CBS's) by not mandating clock synchronization to work (depicted in figure 8). This shaper operates by employing a leaky bucket algorithm, effectively achieving zero congestion loss without requiring time synchronization. In this approach, the ATS monitors a shaping rate and a credit counter, whose value adjusts based on the length of the shaped frames entering the process through a reception port.

As soon as a packet enters a reception port in a bridge, it is directed to the shaped queue preceding the main queue of a specific transmitting port if its length is less than or equal to the maximum length of a shaper packet. Consequently, all flows from a single input port to the same output port share a common queue.

Utilizing the credit and shaping rate as mentioned earlier, the ATS facilitates the transition from the shaped queues to the 802.1Q queues (associated with the transmitting ports) by determining a packet's eligibility for transmission. Eligibility is achieved when the current credits equal or surpass the length of the packet. Upon meeting this criterion, the credit counter is updated and subsequently decremented by the packet's length.

5) FRAME PREEMPTION (FP)

Closing the description on the latency-aware main features of TSN, the IEEE 802.1Qbu standard introduces frame preemption (FP), as depicted in figure 10. This standard establishes two distinct classes of queues: an "express frames" queue class and a "preemptable frames" queue class. In the "express frames" queue class, frames are promptly transmitted upon readiness and have the ability to fragment preemptable frames currently in transmission, unless frame's length is less than or equal to 143 bytes [20]. However, this process does introduce some overhead in relation to preemptable frames. Once the transmission of "express frames" is completed, the transmission of preempted traffic resumes. This implies that frame preemption permits prioritized traffic (termed "express") to be rapidly processed compared to other traffic. This analogy is commonly referenced in the scientific

⁴<https://www.ieee802.org/1/files/public/docs2016/Qcr-specht-specification-mapping-proposal-0516-v01.pdf>

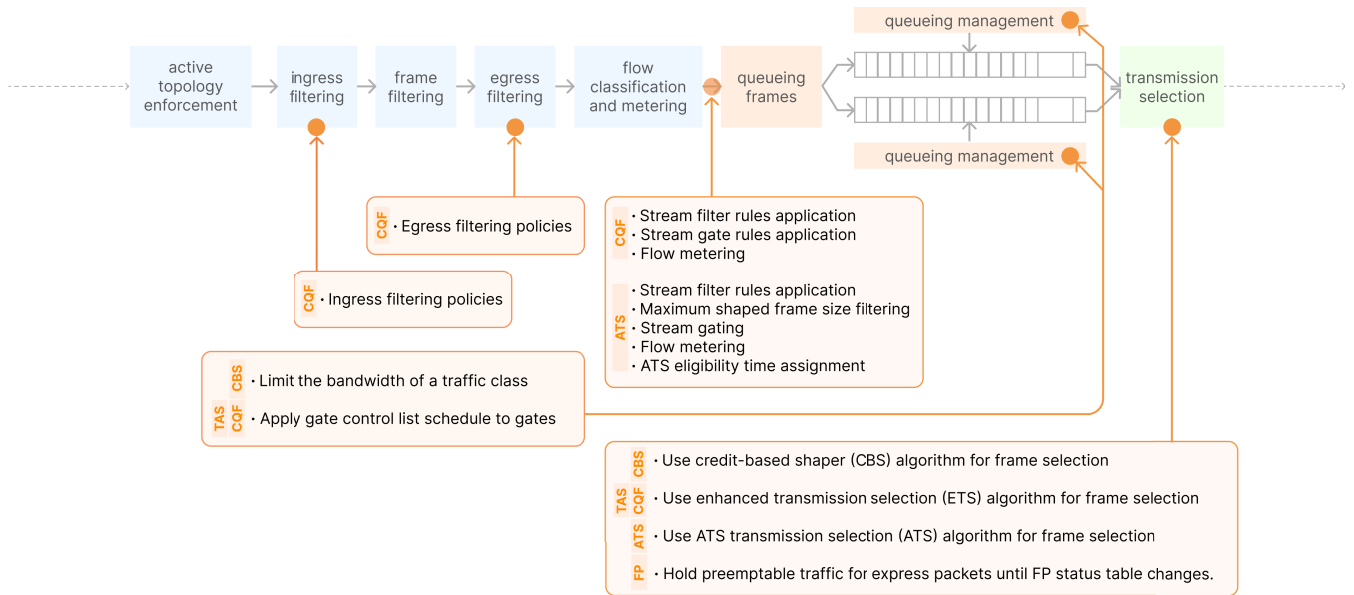


FIGURE 9. TSN's main latency features' impact in detail, along the course of the forwarding process of a bridge.

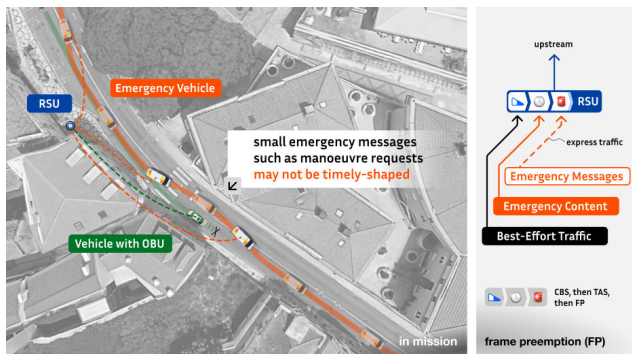


FIGURE 10. Frame Preemption (FP).

community, likening the process to preemptively allocating sand between scheduled rocks, wherein sand represents less critical packets in each forwarding cycle.⁵

6) SUMMARY

Much like the depiction at the end of the previous category, figure 9 provides a detailed representation of the impact of TSN's latency features on the forwarding process of a bridge, adhering to the IEEE 802.1Q standard.

The latency set of functionalities in TSN can almost be seen as the main collection of standards in the entire set of those tools. With the brief description of such standards made in this section, one can verify that not only this category of standards describe how traffic could be shaped, but that traffic use case scenarios in a city could be tackled in order to better

ensure emergency-related flows not to be disturbed by other lower-priority ones, without causing them to stall. Despite being able to identify some standards for the specific use case scenarios in our motivation, the selection of these latency standards do not offer an exclusive way or combination to be a perfect fit for all cases, as they do strictly depend on traffic characteristics such as bandwidth, priority, need for time synchronization, among other features.

Other authors already mentioned these standards in more detailed tutorials, whose reading is recommended as they not only gather more information on their descriptions and considerations, but also attempt to directly model them as a solution to some contexts:

- Deng et al., in a published survey in 2022 [5], have described in detail how does CBS, TAS, ATS, and FP work, as well as they bridged the standards as modeled with AVB considerations to the now hard real-time considerations of TSN. They also performed an end-to-end analysis from AVB to TSN, in which they cover the impact of these latency standards.
- In a more practical sense, in 2023, Ulbricht et al. in [21], proposed a flexible TSN measurement testbed in which they initially cover with a brief but practical-oriented tutorial on some of the latency-related features of TSN such as TAS, CBS, ATS, and CQF.
- In the most recent tutorial article found at time of writing, Walrand in [22] produced a quick tutorial neatly covering the key mechanisms of TSN for shaping and scheduling. This paper was meant for practicing engineers wanting to know more about TSN protocols in a simple and direct manner, but this is a great resource for a very well defined tutorial on how are the latency

⁵<https://www.ietf.org/proceedings/99/slides/slides-99-edu-sessf-newcomers-tutorial-english-language-mike-stjohns-02.pdf>

protocols defined and which considerations should be taken when thinking of combining these mechanisms for a more complex shaping.

E. CONFIGURATION MAIN FEATURES

Finalizing the study of TSN features concepts, the last pillar of TSN is **configuration**. Here two main standards are described (one still as a draft, at time of writing).

1) STREAM RESERVATION PROTOCOL (SRP)

Starting with the stream reservation protocol (SRP), as defined in IEEE 802.1Qat [23] and IEEE 802.1Qcc [24], this is a distributed protocol that, following a peer-to-peer philosophy, specifies admission controls taking advantage of the resource requirements of a flow and available network resources. This way, SRP reserves resources and advertises streams from the talker to the listeners, trying to satisfy QoS requirements per-stream and guarantee the availability of sufficient network resources along the full flow transmission path. SRP uses three signaling mechanisms: the multiple MAC registration protocol (MMRP) for talker registrations; the multiple VLAN registration protocol (MVRP) to declare membership in a VLAN where a stream is being sourced; and the multiple stream registration protocol (MSRP) to reserve network resources.

The SRP mainly works by performing the following steps: 1) advertise a stream from a talker; 2) register the paths along a data flow; 3) calculate the worst-case latency; 4) create a TSN domain by identifying whose nodes are part of a TSN topology; 5) and then reserve the bandwidth. In IEEE 802.1Qcc, SRP was then enhanced with topologies, introducing the concept of TSN architectures and giving TSN topologies of both fully centralized and fully distributed perspectives, differing on the way of conducting and distributing control messages throughout network nodes (both bridges, end-stations, and controllers). A depiction on how could this enhance the smart city infrastructure behavior is represented in figure 11.

2) RESOURCE ALLOCATION PROTOCOL (RAP)

As an evolution of SRP, the TSN workgroup is still drafting, at the time of writing, the resource allocation protocol (RAP) in IEEE 802.1Qdd, which advances SRP in providing support for recent evolution of TSN standardization.⁶ RAP is a stream reservation protocol that extends the capabilities of the latter SRP by providing enhanced service to configure TSN streams that use the QoS functions defined in TSN standards. Working besides a transport layer, as a stand-alone protocol, RAP works in cooperation with a link registration protocol (LRP) that provides port-local service with data transport and database synchronization functions on a point-to-point link.

RAP is backwards-compatible with SRP, and also enhances the interoperability between different TSN

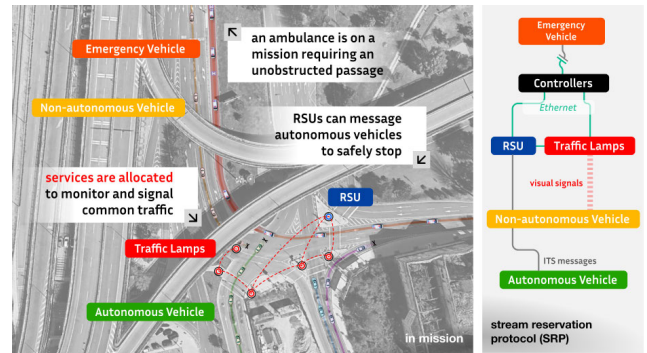


FIGURE 11. Stream reservation protocol (SRP) within a smart city scenario.

protocols such as FRER's. Each stream transmission controlled by the FRER functions also requires that the resources, including bandwidth and buffer space, are reserved at every hop along each of the redundant paths from talker to listener. Thus, RAP is expected to support stream reservation for protocols such as FRER, in which operations of sequencing, splitting and merging of streams will be made possible in the end-devices, in an end-to-end manner. Other interoperability features are thought to be integrated, for easier configuration procedures, such as the reservation for ATS⁷ or CQF.⁸

3) SUMMARY

Ending the exposition of configuration-related features of TSN, one can assess the relevance of these protocols within the scope of hard real-time considerations. Mostly valuable when we are under the scope of a TSN environment deployment in which some bridges will require their schedules (or other configurations) to be applied, SRP or RAP are the two main helpers in attempting to provide ways to more dynamically configure routines having in consideration a proper network topology, in which some nodes can have centralized responsibilities.

As the description above was a brief presentation and contextualization to the concept of SRP and RAP, below we leave references that we strongly advise a reading to know more about this category's standards:

- Despite proposing an alternative solution for admission control to SRP, Álvarez et al. in [25] describes how SRP's admission control can dynamically segregate traffic and reserve bandwidth allowing QoS requirements of real-time services to be guaranteed.
- In more detail on how could RAP achieve the detailed objectives, Osswald et al. in [26] gives an introduction to RAP, including some extensions to it regarding a hybrid configuration model. We recommend this article for anyone aiming to know more about how are

⁷<https://www.ieee802.org/1/files/public/docs2019/dd-chen-RAP-ATS-0619-v02.pdf>

⁸<https://www.ieee802.org/1/files/public/docs2020/dd-finn-CQF-and-shaping-0120-v01.pdf>

⁶<https://www.ieee802.org/1/files/public/docs2017/tsn-chen-RAP-whitepaper-1117-v02.pdf>

resources reserved to ensure QoS requirements in real-time networks. This article focuses on RAP, but also briefly describes the SRP.

- To know more about both protocols (and others) regarding the configuration features of TSN, as RAP is still a draft at time of writing, the reader can also profit from visiting the IEEE workgroup on TSN page⁹ of each one of the standards: the SRP's and the RAP's.

Notwithstanding to these four pillars of TSN as it were described in this section, security is another key feature, whose considerations are transversal to all the mentioned standards and mechanisms. We will not focus our attention on these considerations in this document.

III. FROM COMMON TSN USE CASES, TO THE SMART CITY

TSNs possess inherent capabilities that make them suitable for critical scenarios and deployable environments. Considering "critical software" as a concept in software engineering with strict requirements, TSNs significantly contribute to configuring, implementing, and maintaining architectures that demand such stringent criteria.

Architectural designs incorporating TSN usually prioritize time requirements, but it is important not to exclusively adhere to this principle. Critical systems leveraging TSN standards and mechanisms, not only define requirements and guarantees related to time considerations (such as targeted timings or deadline specifications), but also address bandwidth criteria, ensuring that specified data flow transmissions meet predetermined targets.

As a result, TSN environments find applications in various crucial use cases such as automotive, industry, avionics, railway control, power grids, and similar scenarios where machine-to-machine communication is essential and timing or bandwidth are critical. In this section, we delve into some of these scenarios to better grasp how TSN serves as a relevant solution to these challenges. Figure 12 depicts a simplified representation of all these use cases.

A. AUTOMOTIVE SCENARIO

A vehicle, such as a car, in order it to be driven and to be matched up with criteria of performance, comfort, and safety, needs its components to be able to communicate between them without having anything interfering in such network tasks.

Inside a vehicle's inside computational infrastructure (as depicted in figure 12), several components are displaced and interconnected in a controller area network (CAN) bus topology, under the scope of the SAE J1939 standard, in which constant messages are being transmitted differing on behavior in case of contending when accessing the same bus. These messages combine different types of priorities, which should be taken into consideration in order to ensure the delivery in time and data of some critical content, such as

a triggered message from crash sensor whose consequence could be as vital as triggering the inflation of an airbag. It is important that such a message must not be surpassed by another message with a lower priority, such as from a temperature sensor which could be transmitting data relating high temperatures in a given region of the vehicle.

There are already several protocols and mechanisms which were already designed, implemented, and deployed onto this type of environments to tackle these issues, but they bring one major problem: most of these architectures and mechanisms, although they comply with the necessary certifications, par with lower bandwidth levels of 1 Mbps (CAN) or 5 Mbps (CAN-FD, with flexible data-rate). Additionally, they tend to be highly proprietary, resulting in elevated installation costs, challenges in bug fixing, and adaptation difficulties to accommodate evolving services and requirements over time.

TSN gives out a possible solution to these problems, even considering a combination with CAN [27]. As TSN was created as an extension of Ethernet, tasks such as the wiring installation are automatically reduced in cost. Moreover, the implementation of Ethernet eases the implementation efforts of new services in an era [of Internet of things] when functionalities are being brought very quickly and dynamically to automotive environments [28], such as software-defined vehicles [29]. Along this evolution, from entertainment to control and safety services being introduced into our vehicles, the amount of traffic being transmitted in the in-vehicle network accompanies the same tendency, which brings us very close (if not over) to the limits of our current systems.

Considering Ethernet and TSN, we can answer to both problems of volume of data and installation (and update) costs, allowing to ease the implementation efforts of new services onto this new automotive environment. From the levels stated before for the case of CAN, within the scope of IEEE 802.3, we can now reach bandwidths of 100 Mbps (IEEE 802.3bw), 1 Gbps (IEEE 802.3bp), 10 Mbps (IEEE 802.3cg), or 2.5/5/10 Gbps (IEEE 802.3ch). Looking over the advantages of TSN, we can list some cases in which TSN makes a difference:

- Regarding time synchronization: all the in-vehicle units might answer to a same clock reference, ensuring a constant synchronization between all the components in a same domain. This is essential if one pretends to enable time multiplexing of messages through the inter-component connections;
- Regarding reliability: TSN, with its standards and mechanisms for reliability, has the power to ensure redundancy in the connections made in-vehicle or regarding logic components such as a fallback master clock in case the grand-master fails;
- Regarding latency: communications might start to have specific criteria for latency and jitter values. Sensing equipment, such as the one of the advanced driver assistance systems (ADAS), might have its data properly ensured with time or bandwidth requirements;

⁹<https://1.ieee802.org/tsn>

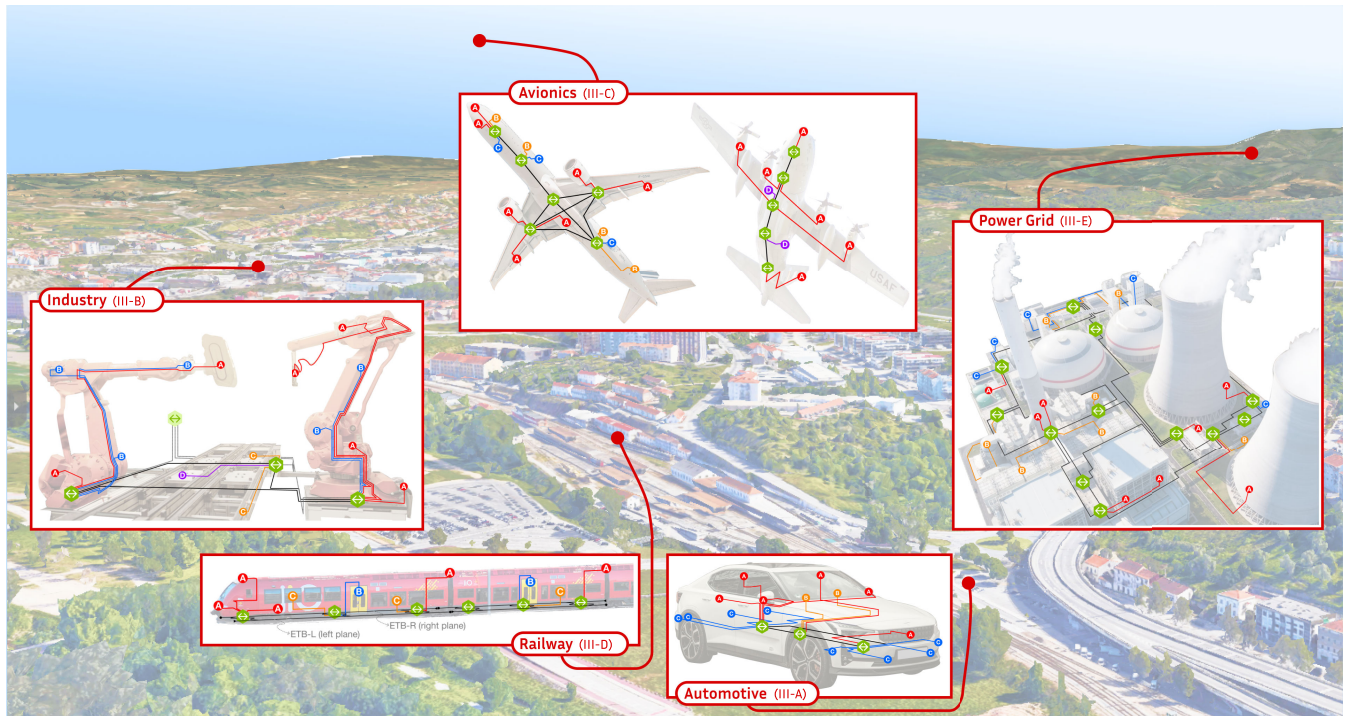


FIGURE 12. Simplified representations of the five most common TSN use cases, all within the context of a city. Aside the backbone connections between TSN bridges represented in green hexagons, letter tagged items identify different traffic flows with different priorities between them.

- Regarding configuration: by delegating responsibilities inside TSN networks, and since these are designed in order to start from pre-configured values, the vehicle can be ready-to-go as soon as it starts, allowing it to start new routines from that moment on, and adapting its rules during its course.

For these reasons, the IEEE TSN workgroup has developed a proper profile for automotive contexts and applications, which is the IEEE P802.1DG standard. Still being a proposal, at time of writing, it tries to define the following issues:

- A specific architecture to the intra-vehicular network, already defining traffic classes, queues, among other characteristics;
- Which TSN functionalities are supposed to be used in cases of latency as well as of reliability, in order to close a little more the possibility of choice between all the available standards and mechanisms, so that unexpected consequences from standards' interactions are diminished in this environment;
- Which functionalities, characteristics, principles, and configurations of TSN should and must be used;
- Some use cases and other future considerations.

B. INDUSTRIAL SCENARIO

Another use case where TSN is currently used as a solution is industry (figure 12). Precisely at the moment Industry 4.0 marks an era as a potential factor for the concepts of smart factory or of cyber-physical systems (CPS), where there is no human-in-the-loop (driving more timing

requirements), TSN gives a real-time communication set of guarantees either of latency or bandwidth [10].

In concept, this scenario is not so different from the previous scenario, since both share the very same characteristic of being considered critical systems. Nonetheless, the industrial case is considerably large regarding the scope of the application of TSN, once that here we also add strict criteria regarding configuration procedures such as the coordination between multiple TSN domains, each one, for instance, with different clock references.

Moreover, the industrial use case also accounts another large responsibility regarding costs. As in the automotive scenario, wiring installation costs are diminished with the adoption of TSN, but here one can also point out the specialized and largely closed equipment industry, which locks down some industries' possibility of risking the development of external solutions. This is an interoperability issue, which TSN increases the chance of solving, since it can bring to industry a large set of developed and tested solutions by more opened communities.

In this line of thought, and alternatively to late (but still in use) solutions such as CAN bus or Modbus (an industrial communication protocol based on a client/server architecture), the Open Platform Communications (OPC) consortium created a unified methodology of communication already described before: the Open Platform Communications unified architecture (OPC-UA). To potentiate such an interoperability, the IEC/IEEE 60802 joint standard allows the interconnection of several buses used in

equipment devices, making this functionality part of the profile here defined jointly by IEEE and IEC for industrial automation.

C. AVIONICS SCENARIO

Almost similar to the automotive use case, there is another scenario in which one can find TSN deployments, that being avionics [30]. In relation to the first scenario, avionics are largely more strict in terms of requirements and necessity for guarantees in time and bandwidth. One can then state that this scenario is a subset of the automotive one, differing in both the effects of disregarding such criteria, and in scale (in number) of its components.

When looking onto aeroplanes, several network architectures are in place according to the type of aircraft (commercial and military aircrafts are depicted in figure 12, in this order).

In a commercial aircraft, the global network architecture is subdivided onto three main domains, these being the passenger information and entertainment service domain, the airline information services domain, and the aircraft control domain. With such a differentiation, it is then relevant to distinguish the high criticality of a message in the aircraft control domain, since it hosts equipment that contributes to the safety of a flight, typically carrying messages from the electronic flight instrument system, engine-indicating and crew alerting system, flight management system, flight controls, among others. This domain, physically, is displaced throughout the aircraft in a system called integrated modular avionics (IMA), in which a general purpose processor is used to host the applications from multiple systems. Here, interoperability is also a key factor to consider, since the network is required to provide an interconnection between the IMA processing, other functions hosted on the network, and data concentrators that provide legacy interfaces.

Through the context of the airline information service domain, non-essential airline operations can be performed, typically also providing a general purpose processing platform and connectivity between the other domains. In terms of criticality, this domain's traffic must have high security requirements, but limited determinism requirements due to non-essential features it supports.

On the other hand, the passenger information and entertainment services domain, as expected, is supposed to have high availability requirements from customer affinity during a flight, and should include supporting the needs of the passengers, which is commonly driven by high performance requirements as well.

Another type of aircraft to consider are military aircrafts, which also use onboard networks to support different functionalities from flight-critical to mission-oriented ones. Having a similar paradigm to commercial aircrafts, military subdivide the network uses into an air vehicle system domain (similar to the aircraft control domain for commercials) and a mission system, being responsible for supporting the varied mission of the aircraft. This domain may include one or more

high-performance mission computers with higher bandwidth than other components in the air vehicle system domain.

Considering these described items, both in the case of the commercial and in the military types of aircrafts, and in a joint work from IEEE and SAE Avionics Networks AS-1 A2, the TSN workgroup is defining a profile for aerospace (IEEE P802.1DP), and both parts of the profile have described some items to consider regarding requirements for these systems:

- Regarding performance, the system must be able to guarantee bandwidth needs of the connected nodes and have strict latency and jitter specifications;
- Aircrafts normally have a longevity of 20 to 30 years old of service, so the deployed system must be resilient in order not to have a large amount of interventions during such a period;
- In terms of availability, interoperability is a key factor to consider in other multiple components from multiple suppliers to be able to communicate between them;
- Regarding reliability, it is critical that the system is able to have an extensive understanding of failure cases such as a loss of a frame, undetected erroneous frames, loss of synchronization, among other events, while it must be able to distinguish and apply different criticalities, segregate different flows according to their QoS class, and support a set of standards defined, such as the no single failure regardless of probability, and others;
- Also described are some specifications on the system flexibility, as the global network of the aircraft must be able to support both synchronous and asynchronous needs, while it may also support the needs of mission systems and flight critical systems on the same network.

All these requirements contribute to foresee time-sensitive networks as a solution to these environments, evolving this use case to another in which one is to find TSN already doing its first steps.

D. RAILWAY CONTROL SCENARIO

As one might have already recognized, we have already taken a look over two types of vehicles, and are now starting to look over the control of another type of vehicle: trains (figure 12). This effectively states TSN as a potential solution, so that machine-to-machine systems (such as vehicular cases) and their interactions are subject to coordination using a similar set of logical rules and standard infrastructures, without disregarding critical conditions.

Railway systems received their first communication standards in 1999, regulated by the IEC, being the train communication network (TCN), as defined in the IEC 61375 standard. This standard allows data exchange between several subsystems, amongst various vendors, and between different types of train carriage types. This intercommunication had already been developed by the International Union of Railways (UIC), in another standard (UIC-556) which described the semantics of variables and messages exchanged between the various components in the system.

This network remains as the base network for current implementations regarding railways scenarios, keeping it as the safe, reliable and robust way to perform in-train communications. Nonetheless, as well as in other use case, the amount of data being transmitted along the carriages, for some years now and in the future, had immensely increased and keeps increasing due to new services and applications. In parallel to this, new routines regarding latency and jitter criteria appear as new requirements to ensure some critical traffic not to be affected by other sharing the same infrastructure.

The basic bus of a train is the wire train bus (WTB) whose specification states a bandwidth of 1 Mbps for the transmission of control messages and commands, which strongly limits new necessities of video transmissions, for instance. This issue was subject to an alternative already standardized, in the IEC-61375-2-5 amendment, creating the Ethernet train backbone (ETB). This alternative allowed that the already standardized communications in the IEC-61375 could now profit from more bandwidth in their data exchanges, considering now a base bandwidth of 100 Mbps from Ethernet connections, but still being capable of interacting with them via proprietary interfaces and protocols for services, or specific to some vehicles that are still much into use nowadays. This makes these systems more complex and less adaptable to future changes and new services to be implemented.

For this reason, the need for a network to be uniform is brought back into discussion, to which TSN is foreseen as a good solution: with TSN one can achieve interoperability and take more advantage from the already existent Ethernet connections, without affecting critical conditions that the current system has as inviolable requirements.

E. POWER GRID SCENARIO

Switching now to a different context than vehicular, and coming back onto an industry-similar context, power grids are the last use case mentioned in this document, but keeping in mind that more use cases exist and will become more relevant in the future (figure 12).

As described in the industry use case, industry-based use cases are an interesting niche to TSN deployments, due to its capabilities of merging operational technologies (OTs) and informational technologies (ITs) in the same media: it is a scenario where it's possible to define, simultaneously, scheduled, best-effort, and reserved traffic.

Energy grids have systems that must follow a certain criteria properly standardized by the IEC 61850 standard, in which proprietary infrastructures and protocols have been replaced by common procedures. While this brought interoperability into these systems, granting connections and data exchanges between vendors, TSN provides infrastructure and enough controllable logic in order to enable, for instance, fault location identification and service restoration (FLISR). Such a service needs predictable low latency to allow the energy distribution to be re-routed from different power grids,

in order to isolate faulted areas and to restore its service to customers without them to notice an interruption during this procedure. This use case is being considered by the Vertical Applications Technical Advisory Group of IEEE, the IEEE 802.24, as a field area network application.¹⁰

To the best of our knowledge, there is only one work which is currently using TSN in order to provide the infrastructure and control, merging OT and IT, located in Granada, Spain, proving this use case [31].

IV. SMART CITY AS A DEPLOYABLE SPACE

By examining the use case scenarios presented in section III, it becomes evident that smart city requirements encompass a wide spectrum, spanning across all of TSN's primary aspects, including time synchronization, reliability, latency, and configuration (or resource management). Expanding on this insight and considering the most common TSN use cases as described in section III, it becomes clear that these individual use cases collectively form a comprehensive and all-encompassing set of applications characteristic of a smart city.

The integration of TSN within the context of a smart city is not a common sight in the literature. While the view of ensuring and improving real-time transmissions over Ethernet is widely debated, its applicability to scenarios with unpredictable and diverse data traffic (which characterizes common smart city use cases) may hint incompatibility with more common TSN environments, as detailed in section III. At the time of writing and to the best of our knowledge, we only found one work that proposes such an integration, within the context of 6G and considering a digital twin version of the city infrastructure, so to robustly facilitate advance analysis, testing, configuration, and system optimizations in a simulated model [2].

As the network traffic from a city poses great challenges to the static and orderly profile of TSN-based environments, there has been some work attempting to solve scheduling problems allowing TSN to be presented as a solution to contribute to the consideration of time in dynamic environments [32], such as smart cities. Several approaches have been presented, from the application of heuristics [33], [34] to a more deterministic view [35], [36] where also urgent but non-isochronous traffic (as emergency traffic in a smart city) would be efficiently and timely transmitted in an enhanced version of TAS.

In this section, we aim to elucidate how a smart city can serve as a central platform that interconnects these diverse use cases, highlighting the potential for TSN integration within other smart city services, with a particular focus on emergency-related features.

A smart city is a concept that has been gaining relevance for some years, as a place where traditional networks and services are made more efficient with the use of digital solutions for the benefit of its inhabitants and

¹⁰<https://www.ieee802.org/24>

businesses.¹¹ Nonetheless, from a wide span of services of a given city, there are plenty items and requirements to consider in unexpected events, which can lead to emergency occurrences, especially looking into the future, when these will be the environments for closed-loop control features and autonomous vehicles [37]. While the point of fully integration and digitization has not been fully reached yet, motivation to achieve this objective is high, with examples such as a smart city deployment in Aveiro, Portugal [38], partially illustrated in figure 13, that already features a substantial number of road-side units (RSUs) denoted by blue hexagons , all of which have the potential to be upgraded or configured to become TSN-capable.

At a scale we are mentioning the context of a Smart City, several classes of traffic are passed from node to node in an already installed network infrastructure, in which parallel emergency and non-critical traffic occurs.

A. TSN IN A SMART CITY: WHAT ARE THE REQUIREMENTS?

These topics in the context of a smart city have not been considered yet by most smart city deployments to the best of our knowledge, and much less has been the application of TSN to tackle them.

To begin with the prospect of an installation or deployment of a TSN approach in a given architecture, one cannot simply think that such a task is at a distance of a set of mere configuration commands. In fact, there should be a reason to deploy such an approach, and its justification should strongly depend on the type of flows one is considering to be subject to such forwarding rules.

To place the reasoning on how to install the TSN approach, we first consider nine different traffic categories in the context of a smart city. Such a list, with a description, is as follows:

- 1) *Best-effort*, as non-sensitive to QoS metrics traffic flows;
- 2) *Network control*, as intermittent but periodical and mostly short-length flows transmitted within the network in order to place commands or to signal configuration procedures;
- 3) *Cyclic traffic*, as a cyclical, but non-isochronous, flow pattern;
- 4) *Audio/voice (in variable bit rate)*, as an example of audio transmission flow, considering a variable bit rate as its characteristic;
- 5) *Isochronous traffic*, as a cyclical with a pre-known, strict and always-on traffic being transmitted within the network;
- 6) *Network monitoring*, as periodical flows of packets used to diagnose a network or network node;
- 7) *Video (in constant bit rate)*, as an example of video transmission flow, considering a constant bit rate as its characteristic;

TABLE 1. Examples of traffic characterization with requirement values for each type of flow.

#	Category	(a)	(b)	(c) ¹³	(d) ¹³	(e)
1	Best-effort	N	–	–	–	0
2	Network control	Y	L	5	60	1
3	Cyclical	Y	L	3	100	2
4	Audio/voice (VBR)	N	–	–	100	2
5	Isochronous	Y	L	2	50	3
6	Network monitoring	Y	L	10	50	3
7	Video (CBR)	Y	B	25	100	4
8	Emergency video (VBR)	N	L/B	–	10	6
9	Emergency event	N	L	–	10	7

(a) Is it periodical? (Y for yes, and N for no).

(b) Guarantee (B for bandwidth, and L for latency).

(c) Cycle time (in ms).

(d) Maximum latency (in ms).

(e) PCP value.

- 8) *Emergency video*, as a critical and high priority traffic flow specific for video transmission of variable bit rate content;
- 9) *Emergency events*, as non-periodical flows of packets with most priority and delivery guarantees on its critical content.

In table 1, one can verify a more defined characterization of the above-mentioned traffic categories. We propose nine traffic categories to encompass the various traffic flows in a smart city deployment. These categories are defined based on periodicity, required guarantees, cycle times, and maximum latencies¹² (if periodicity is applicable). For each of these categories, we assign a priority code point (PCP) value, providing a direct correspondence with a layer 2 VLAN tagging for a specific traffic class.

Keeping these categories in mind, let's proceed to illustrate some traffic flows (F_x) for each category, drawing inspiration from the use case scenarios presented in section III and depicted in figure 12:

- F_1 In all of the use case scenarios, this traffic category can encompass non-prioritized or background traffic;
- F_2 This category is suitable for network protocols that facilitate straightforward and singular configurations, such as routing or switching procedures;
- F_3 This category is pertinent to scenarios with mobility concerns, like case scenarios 1, 2, and 3, where communication stacks in intelligent transport systems, such as ITS-G5, exchange periodically cooperative awareness data about moving vehicles. Additionally, in scenarios like case 2, traffic related to traffic light systems can be characterized as cyclical and intermittent;
- F_4 In the context of use case scenario 4, where sound triggers immediate responses in both city infrastructure and nearby computational nodes, traffic generated for remote audio processing or post-trigger real-time analysis falls under this category;

¹¹<https://smart-cities-marketplace-brochure.eu/2021/>

¹²These values were inspired in both DSCP values (as recommended in RFC 4594) and the 5QI table.



FIGURE 13. Representation of a possible TSN approach adoption in a scenario of a smart city, here inspired on the Aveiro Tech City Living Lab, a smart city deployment in Aveiro, Portugal, accessible via <https://aveiro-living-lab.it.pt> [38]. Some generic services are simplistically represented with letters referencing a same traffic class, and RSUs are marked with blue hexagons , corresponding to units that could be TSN-capable.

- F_5 This category of flows can be designated for communication and control management during emergency events;
- F_6 This category includes health check procedures for network nodes and city elements, bidirectional heartbeat traffic patterns, time synchronization protocol messages, and resource allocation routines;
- F_7 Surveillance cameras in a city, transmitting data to edge or cloud nodes for near-real-time processing and event triggering, fall into this category;
- F_8 This category covers various scenarios, such as the transmission of actuator commands in an ambulance en route to the hospital, guided and commanded by a remote medical actor (use case scenario 2). It also extends to technical assistance with augmented reality in the third use case scenario involving gas leak repair, where real-time data aids a technician's operations;
- F_9 This category accommodates other emergency data that does not require high data throughput. It includes messages from RSUs and other city elements to vehicles, instructing them to divert their paths for approaching emergency response vehicles. In parallel, vital signals from the ambulance's patient in the second use case scenario, as well as reciprocal commands from the remote medical actor to the actuator linked to the under-emergency on-board unit (OBU), are also classified here.

B. ASSESSING STANDARDS FOR THE SELECTED TRAFFIC CATEGORIES

With a deeper understanding of the potential traffic scenarios within a profile of a smart city, we can now explore how TSN presents itself as a viable solution. As elaborated in this document, the TSN standards encompass four primary

pillars, which are time synchronization, reliability, latency, and configuration features. In essence, a smart city use case for TSN would be incomplete without the incorporation of all these fundamental categories.

Regarding **time synchronization**, the smart city profile can take full advantage of the established hierarchy of network and computational nodes within its infrastructure, eliminating the need for the execution of the best master clock (BMC) algorithm. This approach allows to envision that all elements in the city will be precisely synchronized with a centralized time source within the city. Subsequently, RSUs will synchronize with such a time source and act as masters for their connected OBUs. The city-domain time source itself can be synchronized with an external source like GPS, also enabling non-RSU-connected OBUs to achieve synchronization with GPS if necessary [39].

When considering the **latency features** of TSN within the context of the smart city use case, which we view as a comprehensive combination of other common use cases presented in section III, all the TSN latency standards become potential candidates for implementation. However, there are factors to consider when deciding which standard to prioritize.

Referring back to the values provided in table 1, it is prudent to order the critical flows by their maximum latency requirements, from the shortest to the longest. Subsequently, we should explore the potential integration of the credit-based shaper into traffic classes (as identified by the PCP value in table 1), especially when bandwidth guarantees need to be factored into the equation.

The decision to commence with the credit-based shaper in defining the order of our TSN latency tools is driven by its inherent simplicity in engineering. The use of CBS entails the establishment of observation intervals as extensive as

possible to meet the specified latency requirements for the intended flow paths. These observation intervals, as stipulated in section 34.6.1 of the IEEE 802.1Qav standard [16], are recommended not to fall below a threshold of $125\mu\text{s}$, which serves as the minimum value set for the stream reservation class A.

As articulated in the same section, the primary implication of these intervals is their impact on frame size, while the shaper's behavior remains unaffected by this value. In some scenarios, specific combinations of stream bandwidth requirements and transmission port data rates may lead to certain frames being inhibited from continuous streaming. Additional insights into these considerations can be found in Annex L of the IEEE 802.1Qav standard [16].

As we observe that the previously stated requirements for smart city traffic are not fully addressed, we can proceed to the next phase: evaluating the feasibility of employing the time-aware shaper. To assess this potential use, we refer to table 1 and scrutinize which traffic flows necessitate specific latency guarantees. In cases where such requirements exist, the need for TAS becomes evident. An exception arises for flows requiring express delivery and other preempted traffic, where frame preemption could be a suitable solution. The application of time multiplexing brought by TAS can allow, nonetheless, emergency traffic to have a periodically known time window to be preferably used for transmissions.

However, it is crucial to consider and deliberate over the engineering complexities associated with both TAS and FP. While the complexity of TAS design remains reasonable for a single traffic class, challenges magnify as the number of traffic classes multiplies, necessitating diverse strategies to derive optimal schedules. On the other hand, the design complexity for FP is relatively lower, but it still requires attention to fragment overheads. In the case of time-aware shaper, the manner in which schedules are designed can significantly impact the frequency of gate openings and idling.

Regardless of the specific latency-related TSN standards chosen, it is essential to evaluate the per-hop latency. As documented in the research articles mentioned earlier in the scientific community, which represent the current state-of-the-art, one can find various latency analyses for each case. This includes assessments of the application of the asynchronous traffic shaper (ATS), which can serve as a replacement for the CBS in certain scenarios, and evaluations of the cyclic queueing and forwarding (CQF), which complements the TAS through the use of the per-stream filtering and policing.

The substitution of CBS with ATS occurs because ATS ensures a long-term rate of shaper packets, and restricts the number of back-to-back shaper bits that the transmitter can send. This outcome leads to delay bounds within which, by selecting the appropriate parameters for the maximum length of a shaper packet and the shaping rate, both ATS and CBS can provide equivalent bounds on packet delays [22].

The incorporation of frame preemption in the context of a smart city becomes relevant to address the highest-priority and emergency-related data streams. Specifically, the utilization of this TSN mechanism is suitable for a particular class of emergency traffic, specifically the category identified as "emergency event" in table 1. These data packets, associated with emergency events, are typically small in size and irregularly timed, thereby minimizing the likelihood of fragmentation overheads.

Figure 14 illustrates a suggested approach involving the integration of various TSN shapers. This proposal takes into account the PCPs as outlined in table 1.

In figure 14 the CQF standard possibility already initiates the reasoning on what could be done, **reliability-wise**, in the use case of a smart city: with the assessment and application of per-stream filters and policies, one can enhance the more simpler usage of the time-aware shaper adding the possibility of detection and mitigation of disruptive transmissions in the network. Furthermore, we can optimize the reliability features of TSN to introduce time or spatial redundancy, ensuring a high level of availability and responsiveness. This can be achieved by implementing standards such as frame replication and elimination for reliability (FRER), which utilizes redundant paths for replicating stream packets to guarantee content delivery.

Lastly, we must address the importance of the **configuration and resource management capabilities** offered by TSN in the context of a smart city infrastructure and its unique demands. While TSN's SRP resource allocation protocol, with its admission control procedures, is convenient, it has certain limitations for smart city applications, primarily being tailored for AVB scenarios. The introduction of the IEEE 802.1Qcc standard has enhanced SRP by incorporating various network topologies, allowing for more effective utilization of network hierarchy. This enhancement includes the introduction of centralized elements like the centralized network configuration (CNC) and the centralized user configuration (CUC), which play a crucial role in distributing configurations across the extensive network of devices that undergo changes over time.

However, with the current draft of the resource allocation protocol (RAP) standard, the TSN workgroup presents a revised configuration distribution model. This model offers the ability to scale and dynamically reserve streams, with the option of redundancy through FRER. The significance of this implementation lies in its compatibility with the existing SRP methodology. We believe that adopting RAP represents a valuable investment for smart city applications, especially in scenarios where nodes are dynamically relocated, and where a diverse set of features is needed to cater to the requirements of its users at any given time.

The application and update of TSN rules within the context of the smart city network must be granted through configuration protocols of TSN, so that such a deployment is performed without any damage to time synchronization or existing schedules in network devices. Configurations such

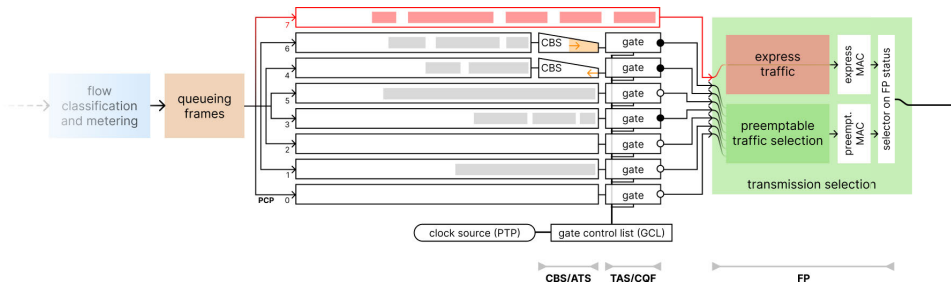


FIGURE 14. Proposal of TSN's latency standardized shapers for the scenario of a smart city, with the described traffic from table 1.

as TAS and CBS (or other latency features of TSN) are also subject to latencies. Following SRP or RAP features of resource management allow these applications to be more scalable within the infrastructure of a smart city.

V. EXPERIMENTAL STUDY

With the concepts presented in sections II-B to II-E in mind and with the analysis done in the previous section, various test scenarios can be devised to showcase the importance and subsequent beneficial gains of the implementation of TSN functions.

A. TESTING SCENARIOS

Two situations will be considered. First, a sensor that constantly produces periodic traffic with tight latency requirements and constant packet size. The application that consumes this traffic is considered hard real-time, and therefore, must receive the data produced by the sensor in the specified moments in time. Second, a camera produces video traffic with a constant bit rate and packet size. The application that consumes this traffic is a soft real-time application, as it can tolerate some missed deadlines and values, with the constant feed of information with a few missed transmissions in between. Finally, in both situations, the traffic produced must coexist with best-effort traffic in the network.

In the first scenario, one of the TSN functions that make the most sense implementing is TAS which, by taking into account the characteristics explained in section II, provides the possibility of a safer coexistence between best-effort and periodic time-sensitive traffic. In the second scenario, the TSN function utilized is the CBS, as it limits bursts, prevents the starvation of lower-priority classes, and limits jitter. This shaper is meant to be used in more relaxed latency-bound traffic, which makes it ideal for the soft real-time application.

For both situations, the approach is to develop a *talker* application that produces time-sensitive traffic with specific requirements, emulating a water supply leakage detection, with a triggering action. In the first situation where the traffic is periodic, this application produced one packet of information in each of the associated transmission intervals of the TAS. On the other end of the data flow, the *listener* application, working as a city's management event service, receives the data and calculates the inter-packet latency to

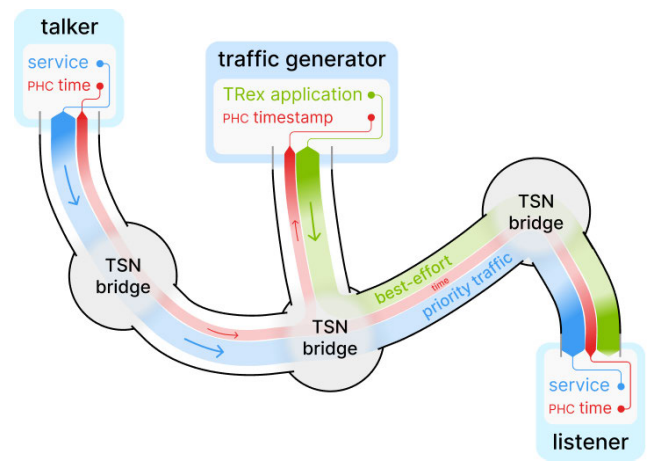


FIGURE 15. Setup of talker and listener through three TSN bridges scenarios, as tested in our laboratories.

measure if the traffic is transmitted as intended. In the second situation, this application emulated a video camera sending video frames at a constant bit rate, which translated into a constant stream of network traffic. In this scenario, no *listener* application is necessary, as the objective is to observe if the *talker* can maintain the constant bit rate while interference traffic is transmitted concurrently.

This setup is depicted in figure 15. The next subsections will further expand the information about these test scenarios and also explain the process necessary to understand and implement the required techniques to ensure the application's requirements.

B. TOOLS USED IN EXPERIMENT

Before delving into the evaluation process it is relevant to mention the various tools and technologies used to achieve the desired implementation. This subsection introduces key TSN technologies and tools applicable to open-source systems based on Linux.

Firstly, the *ethtool*¹³ offers the capability of querying or controlling network drivers and hardware settings. Therefore,

¹³*Ethtool*, a utility for controlling network drivers and hardware. Accessed in April 2024 (<https://mirrors.edge.kernel.org/pub/software/network/ethtool/>)

it was pivotal to determine the capabilities of each network interface card (NIC) used and consequently identify which hardware had the capabilities for supporting time-related features, such as precision time protocol (PTP).

Following with LinuxPTP,¹⁴ this software is a Linux-targeted implementation of PTP, compliant with the IEEE 1588 standard. It allows for deploying and monitoring a PTP-capable device on a network with minimal configurations. It supports several profiles including the gPTP and AVNU automotive profiles. This project provides some key tools to carry out time synchronization:

- `ptp4l`: a daemon that synchronizes the PTP hardware clock (PHC) from the NIC, hardware-agnostic and applies to any 802.1AS-capable NICs with proper device driver support;
- `phc2sys`: a daemon that synchronizes the PHC and the System clock;
- `pmc`: a utility tool to configure `ptp4l` in run-time.

Now regarding hardware-specific technologies, the I210 NIC uses the `igb` driver; this driver offers a feature named *LaunchTime*. This feature allows the Ethernet controller to pre-fetch Ethernet frames from the system memory to the transmission buffer inside the Ethernet medium access control (MAC) controller ahead of its specified transmission time¹⁵. Therefore, this *LaunchTime* technology provides time-deterministic frame transmission and will be essential for future steps in this implementation. This approach bypasses the TCP/IP suite in the Linux networking stack, and therefore, the associated path latency.

Continuing with the regarding the generation of the best-effort traffic, two approaches were possible: (i) generating traffic with a simplistic behavior, i.e. constant bandwidth and packet size with the use of tools such as *iPerf3*¹⁶; or (ii) generating realistic traffic that follows the behavior of the typical traffic of a smart city. The last approach was chosen as it would allow for a more realistic test scenario. Moreover, the tool chosen for this realistic traffic generation was TRex,¹⁷ a fast realistic open-source traffic generation tool, running on standard Intel processors, based on data plane development kit (DPDK), that supports both stateful and stateless traffic generation modes. A key aspect that sets the TRex generator apart is its utilization of DPDK components, which allows for the bypass of performance bottlenecks in the Linux network stack. DPDK is a whole set of libraries and drivers that are used for fast packet processing by allowing the system to exclude the Linux network stack from the packet processing

TABLE 2. Specification of Tested Set Configurations.

Set	Shaper-enabled	LaunchTime-enabled
1	No	No
2	Yes	No
3	Yes	Yes

algorithms so that the tool can interact directly with the network device that is being tested¹⁸¹⁹.

Finally, to ensure confidence in the data collected the most precise method to timestamp the data available to the test must be used. In the TAS scenario the time-stamping is done directly at the NIC and the information is transmitted to the listener application via access control messages (also called ancillary data). In the CBS scenario the data is captured by `tcpdump` which is configured to utilize nanosecond precision when it timestamps the received packets.

C. EVALUATION

The used setup, as depicted in figure 15, consists of two accelerated processing units (APUs) communicating via three TSN-capable switches with an extra APU connected to the middle switch generating best-effort traffic. Moreover, the APUs form a PTP domain in which the application that sends the traffic is the grand-master (GM). In each APU, an instance of `phc2sys` is running: in the GM APU such instance synchronizes the PHC with the system clock; and in the opposite direction, in the slaves, the `phc2sys` instance synchronizes the system clock with the PHC.

The GM APU will produce the time-sensitive traffic that targets the opposite APU. The *talker* application produces the time-sensitive traffic as described in subsection V, while a TRex server produces the interfering best-effort traffic. Both traffic are received by the same APU with the detail that while the best-effort gets discarded, the time-sensitive traffic is consumed by the *listener* application (only in the first scenario).

Moreover, three configurations with increasing optimizations were tested: (i) both shapers TAS or CBS and *LaunchTime* are disabled, to provide the baseline performance for further comparisons as this configuration represented an unoptimized system; (ii) only the associated shaper (TAS or CBS) is enabled, to emphasize the usefulness of the *LaunchTime*; (iii) both the shaper (TAS or CBS) and the *LaunchTime* are enabled, which is expected to provide the best performance in all of the environments. These configurations are represented in table 2.

Focusing on the tests of the first conducted scenario, the transmission schedule of the TAS is configured as it appears in figure 16, where a transmission queue round-robin approach was chosen for testing purposes. In this schedule,

¹⁴The Linux PTP Project official website. Accessed in April 2024 (<https://linuxptp.sourceforge.net/>)

¹⁵Intel Ethernet controller I210 datasheet. Accessed in May 2024 (https://cdrdv2-public.intel.com/333016/333016-I210_Datasheet_v_3_7.pdf)

¹⁶iPerf - The ultimate speed test tool for TCP, UDP and SCTP. Accessed in April 2024 (<https://iperf.fr/>)

¹⁷TRex Low-Cost, High-Speed Stateful Traffic Generator, by Cisco. Accessed in April 2024 (<https://trex-tgn.cisco.com/>)

¹⁸TRex Frequently Asked Questions. Accessed in May 2024 (https://trex-tgn.cisco.com/trex/doc/trex_faq.html#_general)

¹⁹Cisco TRex Traffic Generator: Running Load Tests on a Network. Accessed in May 2024 (<https://promwad.com/news/cisco-trex-traffic-generator>)

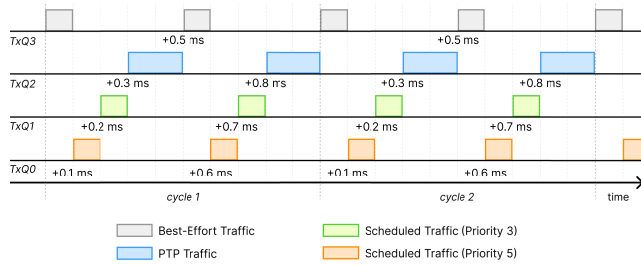


FIGURE 16. TAS Transmission Schedule.

the duration of the transmission cycle is 1 ms, and four types of traffic will be transmitted, each one in a different transmission queue (TxQ): the best-effort traffic is mapped to TxQ 3, the PTP frames are mapped to TxQ 2, the priority 3 traffic is mapped to TxQ 1, and finally the priority 5 traffic is mapped to TxQ 0. Each type of traffic will have two transmission windows in one cycle. The best-effort, priorities 3 and 5 traffic transmission windows will last for 0.1 ms, and the PTP frames transmission windows will last for 0.2 ms. The cycle will start at a specified base time, which will be defined by adding 2 minutes to the instant the configurations are performed.

On the other hand, the tests in the second scenario consist of configuring the CBS in each TSN bridge. The values in this configuration are estimated taking into account the guidelines defined in IEEE 802.1Q [3] and the characteristics of our *talker* application.

Starting with the *talker* application used, it sends a traffic of 8000 packets (p) per second, each with size, λ_{packet} , of 1250 bytes and, therefore, a maximum frame size of equal value (λ_{frame}), in a port with a maximum transmission rate, r_{tx} , of 1000 Mbps. Therefore, the following variables can be defined which will be used in the following expressions:

$$\begin{aligned} p &= 8000 \text{ packets} & \lambda_{\text{packet}} &= 1250 \text{ bytes} \\ \max(\lambda_{\text{frame}}) &= 1250 \text{ bytes} & r_{\text{tx}} &= 1000 \text{ Mbps} \end{aligned} \quad (1)$$

Moreover, the last variable needed to be defined for the required CBS calculations is the maximum interference size, $\lambda_{\text{interference}}$, which represents the size of any burst of traffic that can delay the transmission of a frame that is available for transmission for the protected traffic class. This maximum interference size will therefore be 1500 bytes, i.e. $\max(\lambda_{\text{interference}}) = 1500 \text{ bytes}$.

Starting with the idle slope, s_{idle} , with its calculation in (2), this is the interpretation of the definition of the operating idle slope (stated in the standard as `operIdleSlope`), as the actual bandwidth, in bits per second, that is currently reserved for use by the queue associated with the protected traffic class [3]. Therefore, this bandwidth is calculated by multiplying the number of packets being transmitted each second by the packet size resulting in equation (2). Finally, the idle slope represents the rate of change of credit, in bits per second, when the value of a credit is increasing (i.e., while the protected traffic class is not transmitting). Moreover, the idle

slope value cannot exceed the port transmission rate, r_{tx} [3].

$$s_{\text{idle}} = p \times \lambda_{\text{packet}} \times 8 = 80\,000\,000 \text{ bps} \quad (2)$$

This results in an idle slope value of 80 000 000 bits per second (80.00 Mbps), which in a gigabit Ethernet port represents approximately 8% of reserved bandwidth for the protected traffic class.

Following we have the send slope computation, s_{send} , achieved with the expression in (3), as specified in [3]. Such a parameter represents the rate of change of credit, in bits per second, when the value of credit is decreasing (i.e., while the protected traffic class is transmitting).

$$s_{\text{send}} = s_{\text{idle}} - r_{\text{tx}} = -920\,000\,000 \text{ bps} \quad (3)$$

Regarding the high credit value parameter, c_{high} , this is the maximum value that can be accumulated as credit, and is determined by the worst case interfering traffic as follows in (4) [3].

$$c_{\text{high}} = \max(\lambda_{\text{interference}}) \times 8 \times \frac{s_{\text{idle}}}{r_{\text{tx}}} = 960 \text{ credits} \quad (4)$$

Finally, the low credit value parameter, c_{low} , is the minimum value that can be accumulated as credit, being a function of the values of maximum frame size, port transmission rate, and of the send slope parameter, as described in (5) [3].

$$c_{\text{low}} = \max(\lambda_{\text{frame}}) \times 8 \times \frac{s_{\text{send}}}{r_{\text{tx}}} = -9\,200 \text{ credits} \quad (5)$$

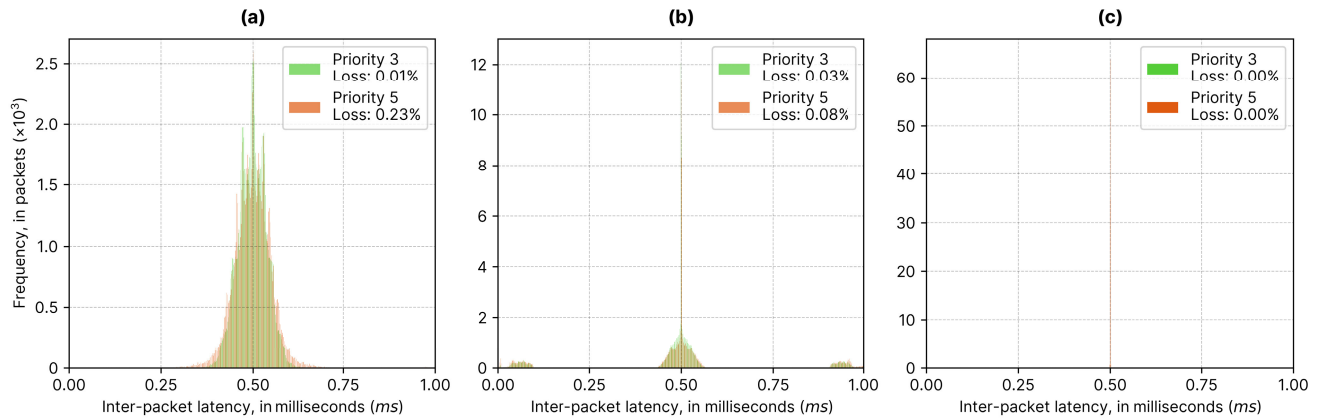
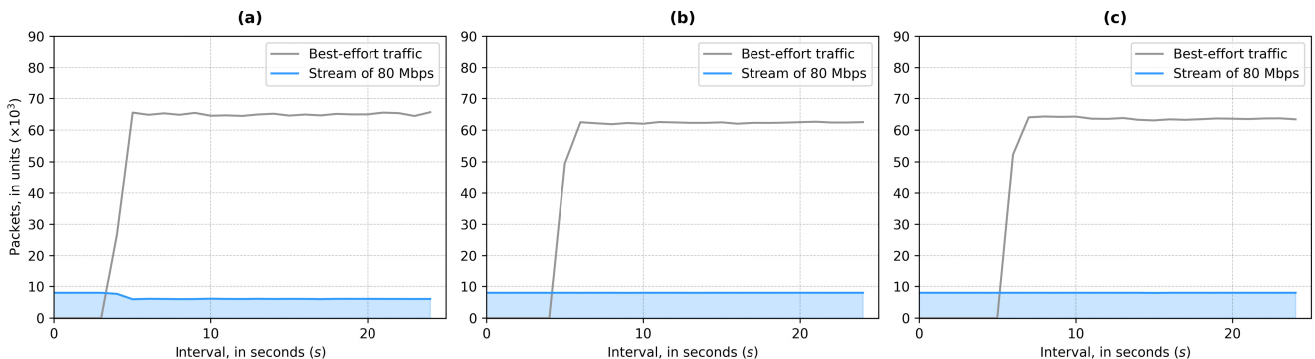
D. RESULTS

All of the results obtained in the first scenario can be seen in table 3 whose plots are depicted in figure 17. It is possible to observe that, in the first developed tests (first scenario depicted in figure 17a), without any form of traffic shaping, the periodic traffic is not being received periodically, that is, despite the packets being sent anywhere within their respective windows, the packets are not leaving the NIC within such a window. Both the loss percentage and the distribution on the histogram are fairly similar for both priorities.

The results obtained in the second test of the first scenario can be seen in figure 17b. It is possible to observe that with the time-aware shaper, the transmission interval of the periodic traffic is being imposed, with some outliers in the edges of the histogram. The majority of the traffic is being kept between the 0.4 ms and 0.6 ms interval, which is expected since the only condition is for the packet to leave the TSN bridge anywhere within the beginning or the end of the transmission window. If a packet is sent at the beginning of the first window and the second packet is also sent at the beginning of the second window, this will result in a difference of 0.5 ms. If the second packet is sent at the end of the second window, the difference will be of 0.6 ms. If a packet is sent at the end of the first window and the second packet is sent at the beginning of the second window, this will result in a difference of 0.4 ms. If the second packet is sent at the end of the second window, the difference will be of 0.5 ms. This

TABLE 3. Detailed results of the first scenario tests, using the time-aware shaper.

Test	Priority	Samples	Mean (ms)	Stddev (ms)	P99 (ms)	Max (ms)	Min (ms)	Loss (%)
1	3	99 991	0.500	0.039	0.589	1.086	0.254	0.01
	5	99 774	0.501	0.054	0.630	1.148	0.137	0.23
2	3	99 964	0.500	0.210	1	5.463	0.001	0.03
	5	99 921	0.500	0.269	1.034	7.500	0.001	0.08
3	3	99 999	0.500	0.005	0.501	0.999	0.098	0
	5	100 000	0.500	0.003	0.501	0.901	0.099	0

**FIGURE 17.** Results of the first scenario tests, using the time-aware shaper. (a), Periodic traffic without time-aware shaping - as test 1. (b), Same traffic with time-aware shaper enabled - as test 2. (c), Periodic traffic with a *LaunchTime*-enabled time-aware shaper - as test 3.**FIGURE 18.** Results of the tests for the second scenario, using the credit-based shaper. (a), Bandwidth-bound traffic not subject to a traffic shaper - as test 1. (b), Same traffic subject to the credit-based shaper - as test 2. (c), Traffic subject to the *LaunchTime*-enabled credit-based shaper - as test 3.

results in an interval of values between 0.4 ms and 0.6 ms. The outliers can be explained by the fact that some packets miss their transmission window and will be re-transmitted in the following window, resulting in at least 1 ms interval between each packet. In the same window, another packet will be sent, resulting in a difference inferior to 0.1 ms. Both the loss percentage and the distribution in the histogram are fairly similar for both priorities. The loss percentage dropped significantly when compared with the first test.

The results obtained in the third test of the first scenario can be seen in figure 17c. It is possible to observe that, with the *LaunchTime* feature, the packets are being sent at the exact instance as specified by the application, resulting

in a single column histogram with data positioned exactly at the tick of 0.5 ms, as it is the exact difference between the middle point of two transmission windows. For example, with priority value of 5 the difference between 0.65 ms and 0.15 ms is of 0.5 ms. Both the loss percentage and the distribution in the histogram are essentially equal for both priorities, with the talker application achieving a 0% loss percentage.

When analyzing the results shown in table 3, it is possible to verify that in all performed tests the average is placed around the 0.5 ms (500 μ s), as expected. The standard deviation is also much lower in the last test, since it has the least variation when comparing with the latter two.

TABLE 4. Results of the CBS experimentation tests (values are in packets per second, pps).

Test	Mean	Median	Stdev	P99
Baseline	8 000	8 000	0.573	8 001
1	6 427	6 061	756	8 001
2	7 998	8 000	3	8 001
3	7 736	8 000	9	8 001

The results regarding the execution of our second scenario can be seen in detail in table 4 and also depicted in figure 18. The objective of this test is to check that the *talker* application used in this scenario is able to meet the 8000 packets per second specification as stated before. With the analysis of this network test profile and of table 4, it is possible to observe that this specification was met, as the average of the packets per second is the specified 8000 units.

Regarding the subsequent tests and starting with a first test, as depicted in figure 18a, we observe that the *talker* application fails to reach the desired throughput, resulting in an average of 6427 packets per second (pps), not covering the original traffic pattern signature of the baseline traffic profile. Moreover, this test had the highest standard deviation of 756 pps showcasing a large variation between bursts. Following are results obtained in the second test, depicted in figure 18b. In this test the credit-based shaper was active, and its effects become highly noticeable when compared with the first test. These effects resulted in the highest throughput which averaged 7998 packets per second, and the lack of variation between bursts resulted in a lower standard deviation which was reduced from 756 to 3 pps. This clearly shows that, with a properly configured shaper like CBS, this application can have a much smoother operation in the same network conditions.

Finally, the results obtained in the final test are depicted in figure 18c. In this test the CBS and the *LaunchTime* feature were enabled, which resulted in results nearly identical to the previous test. Moreover, this effect resulted in the second best throughput of the three tests, 7736 pps. Additionally, the standard deviation also remained very low, this time with the value of 9 pps in all of the three tests. These results not only showcase the importance of the CBS but also that features like the *LaunchTime* are not as relevant to applications that only want to maintain a steady throughput as applications that require precise transmission of packets in time.

Independently from the test scenario, it becomes apparent the necessity and impact of adopting TSN standards (network side) and hardware techniques (application side). By adding these optimizations, the results increasingly improved, and therefore the performance of the system as a whole also became more deterministic. This performance gain also reveals the performance lost in non-optimized systems, figures 17a and 18a, which is usually invisible to the applications and can exacerbate problems in the system.

Moreover, when comparing the adoption of TSN standards and hardware techniques, it is also clear that, by using both optimizations, we could extract most of the possible gain

in performance of the system. However, the adoption of TSN standards should take precedence as it has a wider range of possibilities and effects, and affecting the network that adopts them will possibly have a greater impact on the network performance. This comes in contrast to the hardware technique optimizations because, despite its positive effects are clear, they could not be available (the system hardware might not support them) or even not be compatible with the application implementation.

The impact of adopting TSN, considering the best precision possible, implies some costs as a deployable solution to a smart city infrastructure as dedicated hardware is required. Nonetheless, software-defined solutions, as seen in both figures 17b and 18b, can also result in great outcomes in regards to the precision and, ultimately, to the network traffic organization as described later in section IV.

VI. CONCLUSION

From the discussion of results in the last section, one can experience, in the most practical sense, the contributions that a TSN-based solution can offer to a smart city environment in which several services coexist with different sets of requirements and traffic guarantees needed (of latency and bandwidth). Moreover, and as also seen in the latter section, not only the TSN protocols are beneficial to our line of thought, but also some hardware technologies are necessary to enhance the precision of these techniques, which would be profitable even in smart city use case scenarios, as with more fine control it is possible also to have less constraints regarding how one use the available communication resources (the better and more finely tuned we use our network resources, more is the clearance anyone has to perform other flows and services).

These conclusions help smart cities to pave the way for TSN approaches and protocol implementations as a solution to time considerations with mixed-criticality. This usage and envision of TSN, to build up in a smart city as a solution to properly ensure specific traffic and service conditions in pre-classified traffic, can be a first big step into solutions that could finally give a sense of criticality and time within an urban scenario, as a close-future hybrid environment of autonomous vehicles, interconnected things, and legacy entities. Indeed, with this solution in mind, it is possible to oversee the smart city both as a superset use case for TSN, and as a plausible future TSN profile.

Smart city deployments usually require hardware and software to be scalable and, unfortunately, there is still lack for a comprehensive set of opened implementations to the system designer to profit from regarding what can be used, for instance, in Linux systems and virtualization technologies, which currently carries some scalability issues in regard to the deployment of configurations. We do not pretend to state that is unrealistic to use TSN in the near-future (in this document we have demonstrated the application of some Linux implementations for PTP, TAS, and CBS), but there are still several opportunities for implementations to be done

for other protocols, as well as alternatives to the ones that are already executed.

From this point on, a starting point for a set of new research points is drawn. Besides the design and implementation of open-source alternatives for TSN protocols and mechanisms (for Linux-based systems and others), from this document's work it is possible to: (i) extrude reasonings on how network configurations could be dynamically set in such topologies (a time resource allocation protocol could be designed to be able to ensure specific time-sensitive rules and schedules throughout a set of bridges or networks); (ii) reliability and redundancy considerations could be thought within the context of a smart city; (iii) the interconnection between wired and wireless nodes could also profit from TSN in a transversal way (currently TSN is still a work in progress regarding wireless media [40], and after such a study, a way of sharing different time domains is a challenging task to perform between fixed and mobile nodes); (iv) both inter- and intra-vehicle domains in a smart city could interact so that the communication to and from the city can be conveniently delivered to every entity in the same environment; (v) and lastly, services within a smart city, if virtualized, could be instantiated just-in-time and with a shared time domain, as well with strict computing, network, and timing requirements in place. All of these covered future work topics can be revisited and followed through figures 3, 5, 6, 10, and 11, where smart city functions can be developed from this point on, with time and bandwidth considerations.

REFERENCES

- [1] V. Gavriluț, A. Pruski, and M. S. Berger, "Constructive or optimized: An overview of strategies to design networks for time-critical applications," *ACM Comput. Surveys*, vol. 55, no. 3, pp. 1–35, Feb. 2022, doi: [10.1145/3501294](#).
- [2] Y. Yigit, H. Ahmadi, G. Yurdakul, B. Canberk, T. Hoang, and T. Q. Duong, "Digi-infrastructure: Digital twin-enabled traffic shaping with low-latency for 6G smart cities," *IEEE Commun. Standards Mag.*, vol. 8, no. 3, pp. 28–34, Sep. 2024, doi: [10.1109/MCOMSTD.0002.2300027](#).
- [3] *IEEE Standard for Local and Metropolitan Area Network—Bridges and Bridged Networks*, IEEE Standard 802.1Q-2018, 2018, pp. 1–1993.
- [4] L. Maile, K.-S. Hielscher, and R. German, "Network calculus results for TSN: An introduction," in *Proc. Inf. Commun. Technol. Conf. (ICTC)*, May 2020, pp. 131–140, doi: [10.1109/ICTC49638.2020.9123308](#).
- [5] L. Deng, G. Xie, H. Liu, Y. Han, R. Li, and K. Li, "A survey of real-time Ethernet modeling and design methodologies: From AVB to TSN," *ACM Comput. Surveys*, vol. 55, no. 2, pp. 1–36, Feb. 2023, doi: [10.1145/3487330](#).
- [6] S. Al-Sultan, M. M. Al-Doori, A. H. Al-Bayatti, and H. Zedan, "A comprehensive survey on vehicular ad hoc network," *J. Netw. Comput. Appl.*, vol. 37, pp. 380–392, Jan. 2014, doi: [10.1016/j.jnca.2013.02.036](#).
- [7] A. Nasrallah, A. S. Thyagaturu, Z. Alharbi, C. Wang, X. Shao, M. Reisslein, and H. ElBakoury, "Ultra-low latency (ULL) networks: The IEEE TSN and IETF DetNet standards and related 5G ULL research," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 1, pp. 88–145, 1st Quart., 2019, doi: [10.1109/COMST.2018.2869350](#).
- [8] Z. Satka, M. Ashjaei, H. Fotouhi, M. Daneshmand, M. Sjödin, and S. Mubeen, "A comprehensive systematic review of integration of time sensitive networking and 5G communication," *J. Syst. Archit.*, vol. 138, May 2023, Art. no. 102852, doi: [10.1016/j.sysarc.2023.102852](#).
- [9] S. Rodrigues and J. Lv, "Synchronization in time-sensitive networking: An introduction to IEEE std 802.1AS," *IEEE Commun. Standards Mag.*, vol. 6, no. 4, pp. 14–20, Dec. 2022.
- [10] T. Zhang, G. Wang, C. Xue, J. Wang, M. Nixon, and S. Han, "Time-sensitive networking (TSN) for industrial automation: Current advances and future directions," *ACM Comput. Surveys*, vol. 57, no. 2, pp. 1–38, Oct. 2024, doi: [10.1145/3695248](#).
- [11] *IEEE Standard for Local and Metropolitan Area Networks—Frame Replication and Elimination for Reliability*, IEEE Standard 802.1CB-2017, 2017, pp. 1–102.
- [12] *IEEE Standard for Local and Metropolitan Area Networks—Bridges and Bridged Networks—Amendment 28: Per-Stream Filtering and Policing*, IEEE Standard 802.1Qci-2017, 802.1Q-2014, 802.1Qca-2015, 802.1Qcd-2015, 802.1Q-2014, 802.1Qbv-2015, 802.1Qbu-2016, 802.1Qbz-2016, 2017, pp. 1–65.
- [13] Y. Seol, D. Hyeon, J. Min, M. Kim, and J. Paek, "Timely survey of time-sensitive networking: Past and future directions," *IEEE Access*, vol. 9, pp. 142506–142527, 2021, doi: [10.1109/ACCESS.2021.3120769](#).
- [14] S. Kehler, O. Kleineberg, and D. Heffernan, "A comparison of fault-tolerance concepts for IEEE 802.1 time sensitive networks (TSN)," in *Proc. IEEE Emerg. Technol. Factory Autom. (ETFA)*, Sep. 2014, pp. 1–8, doi: [10.1109/ETFA.2014.7005200](#).
- [15] A. Grigorjew, S. Geißler, P. Diederich, T. Hofffeld, and W. Kellerer, "Resilience in time-sensitive networking: An overview and open research directions," in *Proc. 13th Int. Workshop Resilient Netw. Design Model. (RNDM)*, Sep. 2023, pp. 1–8, doi: [10.1109/rndm59149.2023.10293047](#).
- [16] *IEEE Standard for Local and Metropolitan Area Networks—Virtual Bridged Local Area Networks Amendment 12: Forwarding and Queuing Enhancements for Time-Sensitive Streams*, IEEE Standard 802.1Qav-2009, 802.1Q-2005, 2010, pp. 1–72.
- [17] *IEEE Standard for Local and Metropolitan Area Networks—Bridges and Bridged Networks - Amendment 25: Enhancements for Scheduled Traffic*, IEEE Standard 802.1Q-2014, 802.1Qca-2015, 802.1Q-2014, 2016, pp. 1–57.
- [18] *IEEE Standard for Local and Metropolitan Area Networks—Bridges and Bridged Networks—Amendment 29: Cyclic Queuing and Forwarding*, IEEE Standard 802.1Qch-2017, 2017, pp. 1–30.
- [19] *IEEE Standard for Local and Metropolitan Area Networks—Bridges and Bridged Networks—Amendment 34: Asynchronous Traffic Shaping*, IEEE Standard 802.1Qcr-2020, 2020, pp. 1–151.
- [20] D. Thiele and R. Ernst, "Formal worst-case performance analysis of time-sensitive Ethernet with frame preemption," in *Proc. IEEE 21st Int. Conf. Emerg. Technol. Factory Autom. (ETFA)*, Sep. 2016, pp. 1–9.
- [21] M. Ulbricht, S. Senk, H. K. Nazari, H.-H. Liu, M. Reisslein, G. T. Nguyen, and F. H. P. Fitzek, "TSN-FlexTest: Flexible TSN measurement testbed (extended version)," 2022, *arXiv:2211.10413*.
- [22] J. Walrand, "A concise tutorial on traffic shaping and scheduling in time-sensitive networks," *IEEE Commun. Surv. Tut.*, vol. 25, no. 23, pp. 1941–1953, 2023, doi: [10.1109/COMST.2023.3275038](#).
- [23] *IEEE Standard for Local and Metropolitan Area Networks—Virtual Bridged Local Area Networks Amendment 14: Stream Reservation Protocol (SRP)*, IEEE Standard 802.1Qat-2010, 802.1Q-2005, 2010, pp. 1–119.
- [24] *IEEE Standard for Local and Metropolitan Area Networks—Bridges and Bridged Networks - Amendment 31: Stream Reservation Protocol (SRP) Enhancements and Performance Improvements*, IEEE Standard 802.1Qcc-2018, 802.1Q-2018, 2018, pp. 1–208.
- [25] I. Álvarez, L. Moutinho, P. Pedreiras, D. Bujosa, J. Proenza, and L. Almeida, "Comparing admission control architectures for real-time Ethernet," *IEEE Access*, vol. 8, pp. 105521–105534, 2020, doi: [10.1109/ACCESS.2020.2999817](#).
- [26] L. Osswald, S. Lindner, L. Wüsteney, and M. Menth, "RAP extensions for the hybrid configuration model," in *Proc. 26th IEEE Int. Conf. Emerg. Technol. Factory Autom. (ETFA)*, Sep. 2021, pp. 1–8, doi: [10.1109/ETFA45728.2021.9613246](#).
- [27] B. Achemlal, D. Martini, J. Migge, and J. Villanueva, "Strategies for end-to-end timing guarantees in a centralized software defined vehicle architecture combining CAN with TSN backbone," presented at Automotive Ethernet Congr., Renault Group and Bosch, Munich, Germany, 2023.
- [28] H. H. Bengtsson and N. Navet, "Time-predictable communication in service-oriented architecture-what are the challenges?" presented at Automot. Ethernet Congr., Volvo Cars, Munich, Germany, 2023.
- [29] P. V. Teixeira, D. Raposo, R. Lopes, and S. Sargento, "Software defined vehicles for development of deterministic services," 2024, *arXiv:2407.17287*.

- [30] C. Mauclair, M. Gutiérrez, J. Migge, and N. Navet, "Do we really need TSN in next-generation helicopters? Insights from a case-study," in *Proc. IEEE/AIAA 40th Digit. Avionics Syst. Conf. (DASC)*, Oct. 2021, pp. 1–7, doi: [10.1109/DASC52595.2021.9594349](https://doi.org/10.1109/DASC52595.2021.9594349).
- [31] J. Lázaro, A. Astarloa, M. Rodríguez, U. Bidarte, and J. Jiménez, "A survey on vulnerabilities and countermeasures in the communications of the smart grid," *Electronics*, vol. 10, no. 16, p. 1881, Aug. 2021, doi: [10.3390/electronics10161881](https://doi.org/10.3390/electronics10161881).
- [32] G. Liu, R. Lin, S. Luo, and X. Wang, "Rescheduling for dynamic traffic in time-sensitive networks," in *Proc. IEEE Int. Conf. Commun. Workshops (ICC Workshops)*, Jun. 2024, pp. 239–244, doi: [10.1109/icc-workshops59551.2024.10615613](https://doi.org/10.1109/icc-workshops59551.2024.10615613).
- [33] N. Reusch, S. S. Craciunas, and P. Pop, "Dependability-aware routing and scheduling for time-sensitive networking," *IET Cyber-Phys. Syst., Theory Appl.*, vol. 7, no. 3, pp. 124–146, Sep. 2022, doi: [10.1049/cps2.12030](https://doi.org/10.1049/cps2.12030).
- [34] Y. Li, Z. Yin, Y. Ma, F. Xu, H. Yu, G. Han, and Y. Bi, "Heuristic routing algorithms for time-sensitive networks in smart factories," *Sensors*, vol. 22, no. 11, p. 4153, May 2022.
- [35] Y. Huang, S. Wang, T. Huang, and Y. Liu, "Cycle-based time-sensitive and deterministic networks: Architecture, challenges, and open issues," *IEEE Commun. Mag.*, vol. 60, no. 6, pp. 81–87, Jun. 2022.
- [36] M. Kim, D. Hyeon, and J. Paek, "ETAS: Enhanced time-aware shaper for supporting nonisochronous emergency traffic in time-sensitive networks," *IEEE Internet Things J.*, vol. 9, no. 13, pp. 10480–10491, Jul. 2022, doi: [10.1109/JIOT.2021.3124508](https://doi.org/10.1109/JIOT.2021.3124508).
- [37] S. K. Jagatheesaperumal, S. E. Bibri, J. Huang, J. Rajapandian, and B. Parthiban, "Artificial intelligence of things for smart cities: Advanced solutions for enhancing transportation safety," *Comput. Urban Sci.*, vol. 4, no. 1, p. 10, Apr. 2024, doi: [10.1007/s43762-024-00120-6](https://doi.org/10.1007/s43762-024-00120-6).
- [38] P. Rito, A. Almeida, A. Figueiredo, C. Gomes, P. Teixeira, R. Rosmaninho, R. Lopes, D. Dias, G. Vitor, G. Perna, M. Silva, C. Senna, D. Raposo, M. Luís, S. Sargento, A. Oliveira, and N. B. de Carvalho, "Aveiro tech city living lab: A communication, sensing and computing platform for city environments," *IEEE Internet Things J.*, vol. 10, no. 15, pp. 13489–13510, Aug. 2023, doi: [10.1109/JIOT.2023.3262627](https://doi.org/10.1109/JIOT.2023.3262627).
- [39] R. Martins, D. Raposo, R. Lopes, P. Rito, and S. Sargento, "Time synchronization in V2X communications," in *Proc. IEEE Veh. Netw. Conf. (VNC)*, May 2024, pp. 25–32, doi: [10.1109/VNC61989.2024.10575956](https://doi.org/10.1109/VNC61989.2024.10575956).
- [40] K. Zambouri, M. Noor-A-Rahim, J. John, C. J. Sreenan, H. V. Poor, and D. Pesch, "A comprehensive survey of wireless time-sensitive networking (TSN): Architecture, technologies, applications, and open issues," *IEEE Commun. Surveys Tuts.*, early access, Oct. 25, 2024, doi: [10.1109/COMST.2024.3486618](https://doi.org/10.1109/COMST.2024.3486618).



RUI EDUARDO LOPES (Member, IEEE) received the M.Sc. degree in computer and telematics engineering from the University of Aveiro, Portugal, in 2019, where he is currently pursuing the Ph.D. degree in computer engineering. He is a Teaching Assistant in programming courses with the Department of Electronics, Telecommunications and Informatics, University of Aveiro. He joined the Instituto de Telecomunicações, Aveiro, in 2016, with research

interests including software-defined networks, time-sensitive networks, emergency networks, vehicular ad-hoc networks, and video transmissions over mobile networks.



DUARTE RAPOSO (Member, IEEE) received the M.Sc. and Ph.D. degrees in information science and technology from the University of Coimbra, in 2012 and 2020, respectively. In 2012, he joined CISUC, as a Researcher, working in industrial wireless sensor networks in the topics of network management and security and exploring state-of-the-art anomaly detection techniques. In 2012, he also joined eneida.io, developing industrial wireless solutions in the fields of oil and gas, mining, and energy. Since 2020, he has been with the Institute of Telecommunications, Aveiro, Portugal, as a Research Assistant in the network architectures and protocols. He is the author/co-author of more than 50 scientific works in international conferences and journals and two national patents. His current research interests include wireless communications, deterministic networks, cellular communications, software-defined networks, and edge computing.



RODRIGO MARTINS (Member, IEEE) received the M.Sc. degree in computer and telematics engineering from the University of Aveiro, in 2023. Currently, he is a Research Assistant with the Network Architectures and Protocols (NAP) Group, Institute of Telecommunications, Aveiro, Portugal, with research interests in time-sensitive networking.



SUSANA SARGENTO (Member, IEEE) is a Full Professor with the University of Aveiro and a Senior Researcher with the Instituto de Telecomunicações, where she is leading the Network Architectures and Protocols Group. She was a Visiting PhD Student with Rice University, from 2000 to 2001; and a Guest Faculty with Carnegie Mellon University, in 2008. She has been leading and involved in several European projects, CMU and MIT-Portugal projects, and in several

PRR agendas on autonomous mobility, space, two-wheels vehicles, tourism, and different testbeds. She has co-founded a vehicular networking company, in 2012, Veniam (www.veniam.com). She was the winner of the 2016 EU Prize for Women Innovators and the Femina 2020 Prize in Science, and one of the nominated of Prize ACTIVA Inspiring Women, in 2021. She was the co-coordinator of the national initiative of digital competences in the research axis INCoDe.2030, belonged to the Evaluation Committee of the Fundo200M (www.200m.pt), and is one of the Scientific Directors of CMU-Portugal Programme. She regularly acts as an Expert for European Research Programs. Her main research interests, with more than 450 publications, are in the areas of self-organized networks, intelligent transportation systems, 5G and beyond networks and services, with two large-scale communication and sensing platforms in Porto and Aveiro, with Aveiro Tech City Living Laboratory.

...